

MODULE-1

#VARIABLE REASSIGNMENT TYPE TRACKING

```
x=eval(input())
```

```
y=eval(input())
```

```
print(type(x))
```

```
print(type(y))
```

#VARIABLE REASSIGNMENT TYPE TRACKING:

```
a=int(input())
```

```
a+=5
```

```
print(a)
```

```
a-=2
```

```
print(a)
```

```
a*=3
```

```
print(a)
```

```
a/=2
```

```
print(a)
```

```
a%=4
```

```
print(a)
```

#EVALUATING DIVISION OPERATION

```
a=int(input())
```

```
b=int(input())
```

```
print(a/b)
```

```
print(a//b)
```

```
a=int(input())
```

```
b=int(input())
```

```
print(a/b)
```

```
print(a//b)
```

#SYSTEM COMPARISON

```
a=int(input())
```

```
b=int(input())
```

```
print(a==b)
```

```
print(a!=b)
```

```
print(a>b)
```

```
print(a<b)
```

```
print(a>=b)
```

```
print(a<=b)
```

#TYPE CONVERSION

```
a=int(input())
```

```
print(type(a))
```

```
print(float(a))
```

#BOOLEAN CONVERSION

```
productId=eval(input())
```

```
price=eval(input())
```

```
isAvailable=bool(input())
```

```
print(type(productId))
```

```
print(type(price))
```

```
print(type(isAvailable))
```

#MULTIPLE VARIABLE DISPLAY

```
userValue=eval(input())
```

```
print(type(userValue))
```

```
a=bool(userValue)
```

```
print(type(a))
```

#VALUE EXCHANGE

```
A=int(input())
```

```
B=int(input())
```

```
A,B=B,A
```

```
print(A,B)
```

#ODD EVEN CHECK

```
N=int(input())
```

```
if N%2==0:
```

```
    print("Even")
```

```
else:
```

```
    print("Odd")
```

#DIRECTION CONFLICT DETECTOR

```
a=int(input())
```

```
b=int(input())
```

```
if (a*b)<0:
```

```
    print("True")
```

```
else:
```

```
    print("False")
```

MODULE-2

#MACHINE COMPARISON

```
a=int(input())
```

```
b=int(input())
```

```
c=int(input())
```

```
if a>b and a>c:
```

```
    print("Machine A produces the highest value")
```

```
elif b>a and b>c:
```

```
    print("Machine B produces the highest value")
```

```
elif c>a and c>b:
```

```
    print("Machine C produces the highest value")
```

```
else:
```

```
    print("All machines have equal production value")
```

#IDENTIFY DAY

```
day=int(input())
```

```
if day==1:
```

```
    print("Sunday")
```

```
elif day==2:
```

```
    print("Monday")
```

```
elif day==3:
```

```
    print("Tuesday")
```

```
elif day==4:
```

```
    print("Wednesday")
```

```
elif day==5:
```

```
    print("Thursday")
```

```
elif day==6:
```

```
    print("Friday")
```

```
elif day==7:
```

```
    print("Saturday")
```

```
else:  
    print("Invalid Day")
```

#NUM EVALUATION SYSTEM

```
n=int(input())  
if n>0:  
    print("Positive")  
elif n<0:  
    print("Negative")  
else:  
    print("Zero")
```

#CURRENT BILL

```
u=int(input())  
if 1<=u<=50:  
    print("Low Usage")  
elif 51<=u<=150:  
    print("Medium Usage")  
elif 150<u<=500:  
    print("High Usage")  
else:  
    print("Invalid input")
```

#BANK TRANSACTION CHECKER

```
w=int(input())  
b=int(input())  
if w%100!=0:  
    print("Invalid input")  
elif b<w+10:  
    print("Insufficient balance")
```

else:

print(b-w-10)

#FINE CALC

a=int(input())

if a<0:

print("Invalid input")

elif a<=50:

print("No fine")

elif a<=70:

print(50)

elif a<=100:

print(100)

else:

print(200)

#LEAP YEAR CHECK

a=int(input())

if a<0:

print("Invalid Year")

elif (a%400==0) or (a%4==0 and a%100!=0):

print("Leap Year")

else:

print("Not a Leap Year")

#BONUS CHECK

a=int(input())

b=int(input())

c=int(input())

if b>100 or b<0:

print("Invalid input")

```
elif b>=90 and c>=5:
```

```
    s=(20*a)//100
```

```
    print(s)
```

```
elif b>=75 and c>=3:
```

```
    d=(10*a)//100
```

```
    print(d)
```

```
else:
```

```
    print("No bonus")
```

```
#COMPARISON
```

```
a=int(input())
```

```
b=int(input())
```

```
print(max(a,b))
```

```
#ENERGY BILLING
```

```
a=int(input())
```

```
if 1<=a<=50:
```

```
    print(a*2)
```

```
elif 51<=a<=150:
```

```
    s=a-50
```

```
    d=s*3
```

```
    print((50*2)+d)
```

```
elif 151<=a<=300:
```

```
    q=a-150
```

```
    r=q*5
```

```
    print((50*2)+(100*3)+r)
```

```
elif a>300:
```

```
    print("Invalid input")
```

MODULE 3:

#SECURITY KEY GENERATION

```
n=int(input())
for i in range(2,n+1):
    prime=True
    for j in range(2,i):
        if i%j==0:
            prime=False
            break
    if prime:
        print(i,end=" ")
```

#DATA PACKET ROUTINE PATTERN

```
n=int(input())
for i in range(n):
    num=1
    for j in range(i+1):
        print(num,end=" ")
        num=num*(i-j)//(j+1)
    print()
```

#CROSS PATTERN CONSISTENCY CHECKER

```
n=int(input())
a=[]
for i in range(n):
    row=list(map(int,input().split()))
    a.append(row)

flag=True

for i in range(n):
```



```

for j in range(n):
    if a[i][j]!=a[j][i]:
        flag=False
if flag:
    print("Symmetric Matrix")
else:
    print("Not a Symmetric Matrix")

```

#ODD NUM GENERATOR

```

n=int(input())
ans=0
for i in range(1,n+1,2):
    ans+=i
print(ans)

```

#SHARING OPTIMIZER

```

a=int(input())
b=int(input())
if a==b:
    print("Invalid input")
    exit()
while b!=0:
    a,b=b,a%b
print(a)

```

#PATTERN

```

n=int(input())
for i in range(1,n+1):
    print(" "*(n-i)+"*(2*i-1)")

```

#PATTERN GENERATE

n=int(input())

for i in range(1,n+1):

 print(" "*(n-i)+"*"*(2*i-1))

for i in range(n-1,0,-1):

 print(" "*(n-i)+"*"*(2*i-1))

#PATTER GENERATOR

n=int(input())

for i in range(n):

 for j in range(n):

 print("*",end=" ")

 print()

#GAME SCORE OPTIMIZATION

n=int(input())

ans=0

for i in range(1,n+1,2):

 ans+=i

print(ans)

#SYNCHRONIZING PROCESS CYCLE

a=int(input())

b=int(input())

x=max(a,b)

while True:

 if x%a==0 and x%b==0:

 print(x)

 break

 x+=1

MODULE-4:

#POWER VALUE GENERATOR

try:

```
n=int(input())
```

```
if n<1 or n>10:
```

```
    print("Invalid input")
```

```
else:
```

```
    print([i*i for i in range(1,n+1)])
```

except:

```
    print("Invalid input")
```

#BACKUP LIST SIMULATOR

```
lst=list(map(int,input().split()))
```

```
value=int(input())
```

```
if len(lst)<2:
```

```
    print("Invalid input")
```

```
else:
```

```
    shallow=lst.copy()
```

```
    deep=lst[:]
```

```
    shallow.append(value)
```

```
    deep.append(value)
```

```
    print(shallow)
```

```
    print(deep)
```

```
    print(lst)
```

#SEAT LOCKUP IN EVENT QUEUE

```
n=int(input())
```

```
seats=tuple(map(int,input().split()))
```

```
t=int(input())
```

```
if t in seats:
```

```
    print(f"Seat {t} is at position {seats.index(t)}")
```

else:

print("Invalid input")

#BUS SEAT VERIFICATION

n=int(input())

if n<=0:

print("Invalid input")

else:

seats=list(map(int,input().split()))

if len(seats)!=n:

print("Invalid input")

elif sorted(seats)==seats:

print("Sorted")

else:

print("Not Sorted")

#QUALITY CHECK

n=int(input())

if n<=0:

print("Invalid input")

else:

p=tuple(map(int,input().split()))

if len(p)!=n:

print("Invalid input")

elif all(x>50 for x in p):

print("Accepted")

else:

print("Rejected")

#GRID CELL INSPECTOR

r,c=map(int,input().split())

```
grid=[]
for i in range(r):
    row=list(map(int,input().split()))
    grid.append(row)
ri,ci=map(int,input().split())
if ri<0 or ri>=r or ci<0 or ci>=c:
    print("Invalid input")
else:
    print(grid[ri][ci])
```

#LIST MODIFIER

```
l=list(map(int,input().split()))
i,n=map(int,input().split())
```

```
if i<0 or i>=len(l):
    print("Invalid input")
    exit()
l1=l.copy()
l2=l[:]
l1[i]=n
l2[i]=n
print(l1)
print(l2)
print(l)
```

#INDEX LOCATOR

```
n=int(input())
l=[]
for i in range(n):
    t=tuple(map(int,input().split()))
```

```
l.append(t)
K=int(input())
if K<0 or K>n:
    print("Invalid input")
    exit()
print(f"Room {K} details: (Number={l[K][0]}, Occupied={l[K][1]}, Total={l[K][2]})")
```

#MARATHON RUNNER RANKING

```
n=int(input())
if n<2:
    print("-1")
    exit()
t=list(map(int,input().split()))
if len(t)!=n:
    print("Invalid input")
    exit()
t=list(set(t))
if len(t)<2:
    print("-1")
else:
    t.sort(reverse=True)
    print(t[1])
```

#TEMPERATURE SAFETY MONITORING

```
N=int(input())
if N<0:
    print("Invalid input")
    exit()
t=tuple(map(int,input().split()))
m=max(t)
```

```
if m<100:  
    print("Safe")
```

```
else:  
    print("Unsafe")
```

MODULE-5:

#ARRANGEMENT COUNT

```
n=int(input())
```

```
def fact(n):
```

```
    if n==0 or n==1:
```

```
        return 1
```

```
    return n*fact(n-1)
```

```
if n>12:
```

```
    print("Invalid input")
```

```
else:
```

```
    print(fact(n))
```

#ENERGY SCORE CALC

```
n=int(input())
```

```
def check(n):
```

```
    if n==0:
```

```
        return 0
```

```
    d=n%10
```

```
    if d>7:
```

```
        return-1
```

```
    s=check(n//10)
```

```
    if s==-1:
```

```
        return-1
```

```
    return s+d*d
```

```
ans=check(n)

if ans==-1:
    print("Invalid input")
else:
    print(ans)
```

#SECRET LOCK VALIDATOR

```
n=int(input())
def prime(n,d):
    if d==n:
        return True
    if n%d==0:
        return False
    return prime(n,d+1)
if n<=1:
    print("Invalid input")
elif prime(n,2):
    print("Accepted")
else:
    print("Rejected")
```

#COMPONENT QUALITY CHECK

```
n=int(input())
if "0" in str(n):
    print("Invalid input")
else:
    while n%2==0:
        n//=2
    while n%3==0:
        n//=3
    while n%5==0:
```



```

        n//=5
    if n==1:
        print("Acceptable")
    else:
        print("Not Acceptable")

```

#FILTERED NUMBER REVERSE

```

n=int(input())
def rev_even(n):
    if n==0:
        return "",False
    s,found=rev_even(n//10)
    if (n%10)%2==0:
        return s+str(n%10),True
    return s,found
ans,found=rev_even(n)
if found:
    print(int(ans[::-1]))
else:
    print("Invalid input")

```

#SIGNAL GROWTH TRACKER

```

def fibonacci(n):
    if n==0:
        return 0
    if n ==1:
        return 1
    return fibonacci(n-1) + fibonacci(n-2)
n=int(input())
if n%9==0:
    print("Invalid input")

```

else:

print(fibonacci(n))

#STAIRCASE TOKEN COLLECTOR

n=int(input())

if n<0:

print("Invalid input")

exit()

def add(n):

if n==1:

return 1

return n+add(n-1)

print (add(n))

#DIGI SIGNAL ENCODER

n=int(input())

def binary(n):

if n==0:

return ""

return binary(n//2)+str(n%2)

if n>255:

print("Invalid input")

else:

print(binary(n))

#DIGITAL CHECKSUM CALC

def add(n):

if n==0:

return 0

```
    return n%10 + add(n//10)
n=int(input())
if n%10==0:
    print("Invalid input")
else:
    print(add(n))
```

#REVERSE A STR USING RECURSION

```
def rev(s):
    if len(s)<=1:
        return s
    return rev(s[1:])+s[0]
s=input()
print(rev(s))
```

#First Arrival Gets the Lead

```
n=int(input())
if n<1:
    print("Invalid input")
else:
    arr=list(map(int,input().split()))
    x=int(input())
    arr.insert(0,x)
    print(*arr)
```

#Finding the Right Spot in the Chain

```
n=int(input())
arr=list(map(int,input().split()))
val=int(input())
pos=int(input())
if pos<1 or pos>n+1:
    print("Invalid input")
```

else:

```
arr.insert(pos-1,val)
```

```
print(*arr)
```

#The Tail Leaves The Line

```
n=int(input())
```

```
arr=list(map(int,input().split()))
```

```
if n<2 or len(arr)!=n:
```

```
    print("Invalid input")
```

else:

```
    print(*arr[:-1])
```

#Removing the Chosen Link

```
n=int(input())
```

```
arr=list(map(int,input().split()))
```

```
pos=int(input())
```

```
if n==1 or pos<1 or pos>n:
```

```
    print("Invalid input")
```

else:

```
    arr.pop(pos-1)
```

```
    print(*arr)
```

#Turning the Chain Around

```
n=int(input())
```

```
arr=list(map(int,input().split()))
```

```
if n<2 or len(arr)!=len(set(arr)):
```

```
    print("Invalid input")
```

else:

```
    arr.reverse()
```

```
    print(*arr)
```

#LAST ONE JOINS THE LINE

try:

```
    n=int(input())
```

```

arr=list(map(int,input().split()))
x=int(input())
if n<1 or len(arr)!=n:
    print("Invalid input")
else:
    print(*(arr+[x]))

except:
    print("Invalid input")
#WHEN THE FIRST ONE STEPS OUT
try:
    n=int(input())
    arr=list(map(int,input().split()))
    if n<=1:
        print("Invalid input")
    else:
        arr=arr[1:]
        print(*arr)
except:
    print("Invalid input")
#LOOKING FOR A FAMILIAR VALUE
n=int(input())
arr=list(map(int,input().split()))
x=int(input())
if n<1 or len(arr)!=n:
    print("Invalid input")
else:
    found=False
    for i in range(n):
        if arr[i]==x:
            print(i+1)

```

```
found=True
```

```
break
```

```
if not found:
```

```
    print("Not found")
```

```
#MEASURING THE ENDLESS WALK
```

```
try:
```

```
    n=int(input())
```

```
    arr=list(map(int,input().split()))
```

```
    pos=int(input())
```

```
    if n<1 or pos>n:
```

```
        print("Invalid input")
```

```
    elif pos==0:
```

```
        print("No loop found")
```

```
    else:
```

```
        print(n-pos+1)
```

```
except:
```

```
    print("Invalid input")
```

```
#LATE ENTRY REMOVAL FROM THE DATA QUEUE
```

```
try:
```

```
    size=int(input())
```

```
    arr=list(map(int,input().split()))
```

```
    n=int(input())
```

```
    if size<1:
```

```
        print("Invalid input")
```

```
    else:
```

```
        index=size-n
```

```
        if 0<=index<size:
```

```
            arr.pop(index)
```

```
        print(*arr)
```

```
except:
```

```

    print("Invalid input")
#THE FINAL LINK LOCKS IN
try:
    n=int(input())
    a=list(map(int,input().split()))
    x=int(input())

    if n<1 or len(a)!=n or len(set(a))!=n:
        print("Invalid input")
    else:
        a.append(x)
        print(*a)
except:
    print("Invalid input")
#CLEARING REPEATS ALONG THE WAY
try:
    n=int(input().strip())
    a=list(map(int,input().split()))
except:
    print("Invalid input")
    exit()

if n<=1 or len(a)!=n or any(a[i]>a[i+1] for i in range(n-1)):
    print("Invalid input")
else:
    res=[a[0]]
    for i in range(1,n):
        if a[i]!=a[i-1]:
            res.append(a[i])
    print(*res)
#FINDING A SEAT IN THE ENDLESS LOOP

```

try:

```
n=int(input().strip())  
a=list(map(int,input().split()))  
x=int(input().strip())  
pos=int(input().strip())
```

except:

```
print("Invalid input")  
exit()
```

if n==0 or len(a)!=n or pos<1 or pos>n+1:

```
print("Invalid input")
```

else:

```
a.insert(pos-1,x)  
print(*a)
```

#BREAKING THE LOOP INTO EQUAL PATHS

```
n = int(input())
```

if n < 2:

```
print("Invalid input")
```

else:

```
arr = list(map(int, input().split()))  
mid = (n + 1) // 2
```

```
for i in range(mid):
```

```
    print(arr[i], end=" ")
```

```
print()
```

```
for i in range(mid, n):
```

```
    print(arr[i], end=" ")
```

#ELIMINATION GAME IN CIRCULAR PLAYER RING

try:


```

n=int(input().strip())
k=int(input().strip())
except:
    print("Invalid input")
    exit()

if n<1:
    print("Invalid input")
else:
    res=0
    for i in range(1,n+1):
        res=(res+k)%i
    print(res+1)
#A NEW LEADER AT THE FRONT
try:
    n=int(input().strip())
    a=list(map(int,input().split()))
    x=int(input().strip())
except:
    print("Invalid input")
    exit()

if n<1 or len(a)!=n or x==0:
    print("Invalid input")
else:
    print(x,*a)
#WHEN THE FINAL LINK FADES
try:
    n=int(input().strip())
    a=list(map(int,input().split()))
except:

```

```
print("Invalid input")
exit()
```

```
if n<2 or len(a)!=n or any(i==0 for i in a):
```

```
    print("Invalid input")
```

```
else:
```

```
    print(*a[::-1])
```

```
#WALKING THE CHAINNIN REVERSE
```

```
try:
```

```
    n=int(input().strip())
```

```
    a=list(map(int,input().split()))
```

```
except:
```

```
    print("Invalid input")
```

```
    exit()
```

```
if n<2 or len(a)!=n or len(set(a))!=n:
```

```
    print("Invalid input")
```

```
else:
```

```
    print(*a[::-1])
```

```
#ORDER INSIDE THE ENDLESS RING
```

```
n = int(input())
```

```
if n < 2:
```

```
    print("Invalid input")
```

```
else:
```

```
    a = list(map(int, input().split()))
```

```
if len(a) != n:
```

```
    print("Invalid input")
```

```
else:
```

```
    flag = True
```

```
for i in range(n - 1):
    if a[i] > a[i + 1]:
        flag = False
        break
if flag:
    print("Sorted")
else:
    print("Not Sorted")
```

#CIRCULAR PACKET PRIORITY ORGANIZER

```
n = int(input())
```

```
if n < 1:
```

```
    print("Invalid input")
```

```
else:
```

```
    a = list(map(int, input().split()))
```

```
if len(a) != n:
```

```
    print("Invalid input")
```

```
else:
```

```
    x = int(input())
```

```
    first = [i for i in a if i <= x]
```

```
    second = [i for i in a if i > x]
```

```
if first:
```

```
    print(*first)
```

```
else:
```

```
    print()
```

```
if second:
    print(*second)
else:
    print()
```

```
#LAST STORE
```

```
import sys
```

```
def solve():
    lines = sys.stdin.read().splitlines()
    lines = [line.strip() for line in lines if line.strip()]
```

```
if not lines:
    print("Invalid input")
    return
```

```
if not lines[0].isdigit():
    print("Invalid input")
    return
```

```
N = int(lines[0])
```

```
if len(lines) - 1 != N:
    print("Invalid input")
    return
```

```
stack = []
```

```
output = []
```

```
for i in range(1, N + 1):
```

```
parts = lines[i].split()
```

```
if not parts:
```

```
    print("Invalid input")
```

```
    return
```

```
command = parts[0].upper()
```

```
if command == "PUSH":
```

```
    if len(parts) != 2 or not parts[1].isdigit():
```

```
        print("Invalid input")
```

```
        return
```

```
    stack.append(int(parts[1]))
```

```
elif command == "POP":
```

```
    if len(parts) != 1:
```

```
        print("Invalid input")
```

```
        return
```

```
    if not stack:
```

```
        output.append("Empty")
```

```
    else:
```

```
        output.append(str(stack.pop()))
```

```
else:
```

```
    print("Invalid input")
```

```
    return
```

```
for line in output:
```

```
    print(line)
```

```
if __name__ == "__main__":
```

```
    solve()
```

```
#EXPRESSION VALIDATION SYSTEM
```

```
import sys
```

```
def solve():
```

```
    input_data = sys.stdin.read().strip()
```

```
    if not input_data:
```

```
        print("Invalid input")
```

```
        return
```

```
    balance = 0
```

```
    for char in input_data:
```

```
        if char == '(':
```

```
            balance += 1
```

```
        elif char == ')':
```

```
            balance -= 1
```

```
        if balance < 0:
```

```
            print("Invalid input")
```

```
            return
```

```
    if balance == 0:
```

```
        print("Valid")
```

```
    else:
```

```
        print("Invalid input")
```

```
if __name__ == "__main__":
```

```
    solve()
```

```
#SHADOWS THAT DOMINATE
```

```
import sys
```

```
def solve():
```

```
    input_data = sys.stdin.read().split()
```

```

if not input_data:
    print("Invalid input")
    return

arr = []
for token in input_data:
    if not token.isdigit():
        print("Invalid input")
        return
    arr.append(int(token))

n = len(arr)
result = [-1] * n
stack = []

for i in range(n):
    while stack and arr[stack[-1]] < arr[i]:
        idx = stack.pop()
        result[idx] = arr[i]
    stack.append(i)

print(*result)

if __name__ == "__main__":
    solve()

#ENCODED REPETITION CHAMBER

import sys

def solve():
    s = sys.stdin.read().strip()
    if not s:

```

```
print("Invalid input")
```

```
return
```

```
stack = []
```

```
current_str = ""
```

```
current_num = 0
```

```
num_formed = False
```

```
for i, char in enumerate(s):
```

```
    if char.isdigit():
```

```
        current_num = current_num * 10 + int(char)
```

```
        num_formed = True
```

```
    elif char == '[':
```

```
        if not num_formed:
```

```
            print("Invalid input")
```

```
            return
```

```
        stack.append((current_str, current_num))
```

```
        current_str = ""
```

```
        current_num = 0
```

```
        num_formed = False
```

```
    elif char == ']':
```

```
        if not stack:
```

```
            print("Invalid input")
```

```
            return
```

```
        prev_str, repeat_count = stack.pop()
```

```
        current_str = prev_str + (current_str * repeat_count)
```

```
    elif char.isalpha():
```

```
        if num_formed:
```

```
            print("Invalid input")
```

```
            return
```

```
        current_str += char
```



```

    else:
        print("Invalid input")
        return

if stack or num_formed:
    print("Invalid input")
else:
    print(current_str)

if __name__ == "__main__":
    solve()

#WAREHOUSE EXPRESSION PROCESSING SYSTEM

import sys

def precedence(op):
    if op in ('+', '-'):
        return 1
    if op in ('*', '/'):
        return 2
    return 0

def solve():
    s = sys.stdin.read().strip().replace('\r', '').replace('\n', '')
    if not s:
        print("Invalid input")
        return

    balance = 0
    for char in s:
        if char == '(':
            balance += 1

```

```

elif char == ')':
    balance -= 1
    if balance < 0:
        print("Invalid input")
        return
if balance != 0:
    print("Invalid input")
    return

postfix = []
operators = []
i = 0
n = len(s)

while i < n:
    char = s[i]

    if char.isdigit():
        num = ""
        while i < n and s[i].isdigit():
            num += s[i]
            i += 1
        postfix.append(num)
        continue
    elif char == '(':
        operators.append(char)
    elif char == ')':
        while operators and operators[-1] != '(':
            postfix.append(operators.pop())
        if operators:
            operators.pop()

```

```

elif char in ('+', '-', '*', '/'):
    while operators and precedence(operators[-1]) >= precedence(char):
        postfix.append(operators.pop())
    operators.append(char)
else:
    print("Invalid input")
    return
i += 1

while operators:
    postfix.append(operators.pop())

print(" ".join(postfix))

if __name__ == "__main__":
    solve()

#MODIFICATION MODULE

import sys

def solve():

    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])

    stack = []
    idx = 1

    for _ in range(n):

```

```

if idx >= len(input_data):
    break

operation_type = int(input_data[idx])
idx += 1

if operation_type == 1:

    x = int(input_data[idx])
    idx += 1
    stack.append(x)

elif operation_type == 2:
    if not stack:
        print("Invalid input")
    else:
        stack.pop()

elif operation_type == 3:
    if not stack:
        print("Invalid input")
    else:
        print(stack[-1])

if __name__ == "__main__":
    solve()

#STACKING PLATES IN A CAFETERIA
import sys

def solve():
    input_data = sys.stdin.read().splitlines()

```

```
lines = [line.strip() for line in input_data if line.strip()]
```

```
if len(lines) < 2:
```

```
    print("Invalid input")
```

```
    return
```

```
try:
```

```
    n = int(lines[0])
```

```
except ValueError:
```

```
    print("Invalid input")
```

```
    return
```

```
if not (1 <= n <= 100):
```

```
    print("Invalid input")
```

```
    return
```

```
plate_tokens = lines[1].split()
```

```
if len(plate_tokens) != n:
```

```
    print("Invalid input")
```

```
    return
```

```
plates = []
```

```
for token in plate_tokens:
```

```
    if not token.isdigit():
```

```
        print("Invalid input")
```

```
        return
```

```
    val = int(token)
```

```
    if not (1 <= val <= 1000):
```

```
        print("Invalid input")
```

```
        return
```

```
    plates.append(str(val))
```

```

print(" ".join(plates))

if __name__ == "__main__":
    solve()

#PROCESSING RECENT ENTRIES

import sys

def solve():
    input_data = sys.stdin.read().splitlines()
    lines = [line.strip() for line in input_data if line.strip()]

    if len(lines) < 3:
        print("Invalid input")
        return

    try:
        n = int(lines[0])
    except ValueError:
        print("Invalid input")
        return

    if not (1 <= n <= 100):
        print("Invalid input")
        return

    actions = lines[1].split()
    if len(actions) != n:
        print("Invalid input")
        return

```

```

action_ids = []

for token in actions:

    if not token.isdigit():

        print("Invalid input")

        return

    val = int(token)

    if not (1 <= val <= 1000):

        print("Invalid input")

        return

    action_ids.append(str(val))


try:

    k = int(lines[2])

except ValueError:

    print("Invalid input")

    return


if not (1 <= k <= 200):

    print("Invalid input")

    return


if k > n:

    print("Invalid input")

    return


remaining = action_ids[:n - k]

print(" ".join(remaining))


if __name__ == "__main__":

    solve()

#TASK DISPLAY

```

```
import sys

def solve():
    input_data = sys.stdin.read().splitlines()
    lines = [line.strip() for line in input_data if line.strip()]

    if not lines:
        print("Invalid input")
        return

    try:
        n = int(lines[0])
    except ValueError:
        print("Invalid input")
        return

    if not (0 <= n <= 100):
        print("Invalid input")
        return

    if n == 0:
        print("Invalid input")
        return

    if len(lines) < 2:
        print("Invalid input")
        return

    elements = lines[1].split()
    if len(elements) != n:
        print("Invalid input")
```



```
    return
```

```
valid_elements = []
```

```
for token in elements:
```

```
    if not token.isdigit():
```

```
        print("Invalid input")
```

```
        return
```

```
    val = int(token)
```

```
    if not (1 <= val <= 1000):
```

```
        print("Invalid input")
```

```
        return
```

```
    valid_elements.append(str(val))
```

```
print(" ".join(valid_elements))
```

```
if __name__ == "__main__":
```

```
    solve()
```

```
#SIGNALS THAT MUST CLOSE
```

```
import sys
```

```
def solve():
```

```
    s = sys.stdin.read().strip().replace('\r', '').replace('\n', '')
```

```
    if not s:
```

```
        print("Invalid input")
```

```
        return
```

```
    balance = 0
```

```
    for char in s:
```

```
        if char == '(':
```

```
            balance += 1
```

```
        elif char == ')':
```

```

        balance -= 1

    if balance < 0:
        print("Not Balanced")
        return

    else:
        print("Invalid input")
        return

if balance == 0:
    print("Balanced")
else:
    print("Not Balanced")

if __name__ == "__main__":
    solve()

#THE LINE OF WAITING SOULS
import sys
from collections import deque

def solve():
    input_data = sys.stdin.read().splitlines()
    lines = [line.strip() for line in input_data if line.strip()]

    if not lines:
        return

    try:
        n = int(lines[0])
    except ValueError:
        return

```

```
queue = deque()
```

```
for i in range(1, len(lines)):
```

```
    parts = lines[i].split()
```

```
    if not parts:
```

```
        continue
```

```
    command = parts[0]
```

```
    if command == "enqueue":
```

```
        if len(parts) == 2:
```

```
            try:
```

```
                val = int(parts[1])
```

```
                queue.append(val)
```

```
            except ValueError:
```

```
                pass
```

```
    elif command == "dequeue":
```

```
        if not queue:
```

```
            print("Queue is empty")
```

```
        else:
```

```
            print(queue.popleft())
```

```
    elif command == "peek":
```

```
        if not queue:
```

```
            print("Queue is empty")
```

```
        else:
```

```
            print(queue[0])
```

```
    elif command == "is_empty":
```

```
        if not queue:
```

```
            print("True")
```

```
        else:
```

```
            print("False")
```

```

        elif command == "size":
            print(len(queue))

if __name__ == "__main__":
    solve()
#THE INVERTED WAITING LINE
import sys

def solve():
    input_data = sys.stdin.read().splitlines()
    lines = [line.strip() for line in input_data if line.strip()]

    if not lines:
        print("Invalid input")
        return

    try:
        n = int(lines[0])
    except ValueError:
        print("Invalid input")
        return

    if not (0 <= n <= 1000):
        print("Invalid input")
        return

    if n == 0:
        if len(lines) > 1:
            print("Invalid input")
        else:
            print("")

```

```
    return
```

```
if len(lines) < 2:
```

```
    print("Invalid input")
```

```
    return
```

```
elements = lines[1].split()
```

```
if len(elements) != n:
```

```
    print("Invalid input")
```

```
    return
```

```
valid_elements = []
```

```
for token in elements:
```

```
    try:
```

```
        val = int(token)
```

```
        if not (-100000 <= val <= 100000):
```

```
            print("Invalid input")
```

```
            return
```

```
        valid_elements.append(str(val))
```

```
    except ValueError:
```

```
        print("Invalid input")
```

```
    return
```

```
valid_elements.reverse()
```

```
print(" ".join(valid_elements))
```

```
if __name__ == "__main__":
```

```
    solve()
```

```
#QUEUE SORTING VALIDATOR
```

```
import sys
```

```
def solve():

    input_data = sys.stdin.read().splitlines()

    lines = [line.strip() for line in input_data if line.strip()]

    if len(lines) < 2:

        print("Invalid input")

        return

    try:

        n = int(lines[0])

    except ValueError:

        print("Invalid input")

        return

    if not (1 <= n <= 10**5):

        print("Invalid input")

        return

    elements = lines[1].split()

    if len(elements) != n:

        print("Invalid input")

        return

    queue = []

    seen = set()

    for token in elements:

        if not token.isdigit():

            print("Invalid input")

            return

        val = int(token)

        if not (1 <= val <= n) or val in seen:
```

```

        print("Invalid input")

        return

    seen.add(val)

    queue.append(val)

stack = []
expected = 1

for item in queue:

    stack.append(item)

    while stack and stack[-1] == expected:

        stack.pop()

        expected += 1

if expected == n + 1 and not stack:

    print("Yes")

else:

    print("No")

if __name__ == "__main__":

    solve()

#METRO CITY FUEL STATION

import sys

from collections import deque

def solve():

    input_data = sys.stdin.read().splitlines()

    lines = [line.strip() for line in input_data if line.strip()]

    if len(lines) < 2:

        print("Invalid input")

```

```
return
```

```
try:
```

```
    n = int(lines[0])
```

```
    m = int(lines[1])
```

```
except ValueError:
```

```
    print("Invalid input")
```

```
    return
```

```
if n <= 0 or m <= 0:
```

```
    print("Invalid input")
```

```
    return
```

```
if len(lines) - 2 < m:
```

```
    print("Invalid input")
```

```
    return
```

```
queue = deque()
```

```
for i in range(2, 2 + m):
```

```
    parts = lines[i].split()
```

```
    if not parts:
```

```
        print("Invalid input")
```

```
        return
```

```
command = parts[0].upper()
```

```
if command == "ARRIVE":
```

```
    if len(parts) != 2:
```

```
        print("Invalid input")
```

```
        return
```



```

try:
    x = int(parts[1])
except ValueError:
    print("Invalid input")
    return

if len(queue) < n:
    queue.append(str(x))
elif command == "SERVE":
    if len(parts) != 1:
        print("Invalid input")
        return
    if queue:
        queue.popleft()
    else:
        print("Invalid input")
        return

if not queue:
    print("Empty")
else:
    print(" ".join(queue))

if __name__ == "__main__":
    solve()

#SMART CITY TRAFFIC CONTROL

import sys

def solve():
    input_data = sys.stdin.read().splitlines()
    lines = [line.strip() for line in input_data if line.strip()]

```

```
if len(lines) < 2:
    print("Invalid Input")
    return

try:
    n = int(lines[0])
except ValueError:
    print("Invalid Input")
    return

if n <= 0 or not (1 <= n <= 10**5):
    print("Invalid Input")
    return

elements = lines[1].split()
if len(elements) != n:
    print("Invalid Input")
    return

arr = []
for token in elements:
    try:
        val = int(token)
        if not (-10**9 <= val <= 10**9):
            print("Invalid Input")
            return
        arr.append(val)
    except ValueError:
        print("Invalid Input")
        return
```

```

if n == 1:
    print("-1")
    return

for i in range(n):
    prev_idx = (i - 1 + n) % n
    next_idx = (i + 1) % n

    if arr[i] > arr[prev_idx] and arr[i] > arr[next_idx]:
        print(arr[i])
        return

print("-1")

if __name__ == "__main__":
    solve()

#TICKET COUNTER MANAGEMENT SYSTEM

from collections import deque

n = int(input())
q = deque()

for _ in range(n):
    s = input().strip().split()

    if len(s) == 2 and s[0] == "ARRIVAL":
        if not s[1].isdigit() or int(s[1]) <= 0:
            print("Invalid input")
            break
        q.append(s[1])

```

```
elif len(s) == 1 and s[0] == "SERVE":
```

```
    if q:
```

```
        print(q.popleft())
```

```
    else:
```

```
        print("No Passengers")
```

```
elif len(s) == 1 and s[0] == "STATUS":
```

```
    if q:
```

```
        print(*q)
```

```
    else:
```

```
        print("Counter Empty")
```

```
else:
```

```
    print("Invalid input")
```

```
    break
```

```
#FIRST NEGATIVE NUMBER IN EVERY WINDOW
```

```
import sys
```

```
def solve():
```

```
    input_data = sys.stdin.read().split()
```

```
    if not input_data:
```

```
        return
```

```
    N = int(input_data[0])
```

```
    if N <= 0:
```

```
        print("Invalid input")
```

```
        return
```

```
    arr = [int(x) for x in input_data[1:N+1]]
```

```
    K = int(input_data[N+1])
```

```

result = []

from collections import deque

neg_indices = deque()

for i in range(K):
    if arr[i] < 0:
        neg_indices.append(i)

if neg_indices:
    result.append(str(arr[neg_indices[0]]))
else:
    result.append("0")

for i in range(K, N):
    while neg_indices and neg_indices[0] <= i - K:
        neg_indices.popleft()

    if arr[i] < 0:
        neg_indices.append(i)

    if neg_indices:
        result.append(str(arr[neg_indices[0]]))
    else:
        result.append("0")

print(" ".join(result))

if __name__ == '__main__':
    solve()

#TECH PARK TASK ROTATION

```

```
import sys

def solve():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    try:
        N = int(input_data[0])
    except ValueError:
        print("Invalid input")
        return

    if N <= 0:
        print("Invalid input")
        return

    if len(input_data) < N + 2:
        print("Invalid input")
        return

    task_ids = []
    for i in range(1, N + 1):
        try:
            task_ids.append(int(input_data[i]))
        except ValueError:
            print("Invalid input")
            return

    try:
        K = int(input_data[N + 1])
```

```
except ValueError:
```

```
    print("Invalid input")
```

```
    return
```

```
if K <= 0:
```

```
    print("Invalid input")
```

```
    return
```

```
if len(input_data) != N + 2:
```

```
    print("Invalid input")
```

```
    return
```

```
survivor_idx = 0
```

```
for i in range(2, N + 1):
```

```
    survivor_idx = (survivor_idx + K) % i
```

```
print(task_ids[survivor_idx])
```

```
if __name__ == '__main__':
```

```
    solve()
```

```
#DATA CENTER COOLING RING
```

```
import sys
```

```
def main():
```

```
    input_data = sys.stdin.read().splitlines()
```

```
    cleaned_lines = [line.strip() for line in input_data if line.strip()]
```

```
if len(cleaned_lines) < 4:
```

```
    print("Invalid input")
```

```
    return
```

```

try:
    n = int(cleaned_lines[0])
    arr_elements = cleaned_lines[1].split()
    k = int(cleaned_lines[2])
    t = int(cleaned_lines[3])
    arr = [int(x) for x in arr_elements]
except ValueError:
    print("Invalid input")
    return

if n <= 0 or k <= 0 or k > n or len(arr) != n:
    print("Invalid input")
    return

extended_arr = arr + arr[:k - 1]

consecutive_count = 0
for num in extended_arr:
    if num > t:
        consecutive_count += 1
        if consecutive_count >= k:
            print("Yes")
            return
    else:
        consecutive_count = 0

print("No")

if __name__ == '__main__':
    main()
#HIGHWAY SERVICE YARD

```



```
import sys

from collections import Counter


def main():

    input_data = sys.stdin.read().splitlines()

    cleaned_lines = [line.strip() for line in input_data if line.strip()]

    if len(cleaned_lines) < 2:

        print("Invalid input")

        return

    try:

        n = int(cleaned_lines[0])

        arr_elements = cleaned_lines[1].split()

        arr = [int(x) for x in arr_elements]

    except ValueError:

        print("Invalid input")

        return

    if n <= 0 or len(arr) != n:

        print("Invalid input")

        return

    counts = Counter(arr)

    threshold = n // 2

    for worker_id, count in counts.items():

        if count > threshold:

            print(worker_id)

            return
```

```

print("-1")

if __name__ == '__main__':
    main()

#BINARY TREE EMPLOYEE HIERARCHY

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def build(arr):
    if not arr or arr[0] == -1:
        return None
    nodes = [None if x == -1 else Node(x) for x in arr]
    for i in range(len(arr)):
        if nodes[i]:
            l = 2 * i + 1
            r = 2 * i + 2
            if l < len(arr):
                nodes[i].left = nodes[l]
            if r < len(arr):
                nodes[i].right = nodes[r]
    return nodes[0]

def inorder(root):
    if root:
        inorder(root.left)
        ans.append(root.data)
        inorder(root.right)

```

```
n = int(input())
```

```
if n <= 0:
```

```
    print("Invalid input")
```

```
else:
```

```
    arr = list(map(int, input().split()))
```

```
    arr = arr[:n]
```

```
    root = build(arr)
```

```
    ans = []
```

```
    inorder(root)
```

```
    print(*ans)
```

```
#SMART WAREHOUSE ROBOT NAVIGATION
```

```
class Node:
```

```
    def __init__(self, x):
```

```
        self.data = x
```

```
        self.left = None
```

```
        self.right = None
```

```
def preorder(root):
```

```
    if root:
```

```
        print(root.data, end=" ")
```

```
        preorder(root.left)
```

```
        preorder(root.right)
```

```
n = int(input())
```

```
if n % 2 == 0:
```

```
    print("Invalid input")
```

```
else:
```

```
    a = list(map(int, input().split()))
```

```
    nodes = [Node(x) for x in a]
```

```

for i in range(n):
    l = 2 * i + 1
    r = 2 * i + 2
    if l < n:
        nodes[i].left = nodes[l]
    if r < n:
        nodes[i].right = nodes[r]
preorder(nodes[0])
#ORGANIZATIONAL DEPTH ANALYSIS SYSTEM
class Node:
    def __init__(self, x):
        self.data = x
        self.left = None
        self.right = None

def build(arr):
    if not arr or arr[0] == -1:
        return None
    nodes = [None if x == -1 else Node(x) for x in arr]
    for i in range(len(arr)):
        if nodes[i]:
            l = 2 * i + 1
            r = 2 * i + 2
            if l < len(arr):
                nodes[i].left = nodes[l]
            if r < len(arr):
                nodes[i].right = nodes[r]
    return nodes[0]

def height(root):
    if root is None:

```

```
        return 0

    return 1 + max(height(root.left), height(root.right))
```

```
n = int(input())
```

```
if n < 0:
```

```
    print("Invalid input")
```

```
else:
```

```
    arr = list(map(int, input().split()))
```

```
    arr = arr[:n]
```

```
    root = build(arr)
```

```
    print(height(root))
```

```
#DYNAMIC EMPLOYEE ADDITION SYSTEM
```

```
from collections import deque
```

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.left = None
```

```
        self.right = None
```

```
def build_tree(arr):
```

```
    if not arr or arr[0] == -1:
```

```
        return None
```

```
    root = Node(arr[0])
```

```
    q = deque([root])
```

```
    i = 1
```

```
    while q and i < len(arr):
```

```
        curr = q.popleft()
```

```
if i < len(arr) and arr[i] != -1:
```

```
    curr.left = Node(arr[i])
```

```
    q.append(curr.left)
```

```
    i += 1
```

```
if i < len(arr) and arr[i] != -1:
```

```
    curr.right = Node(arr[i])
```

```
    q.append(curr.right)
```

```
    i += 1
```

```
return root
```

```
def insert(root, x):
```

```
    q = deque([root])
```

```
    while q:
```

```
        curr = q.popleft()
```

```
        if curr.left is None:
```

```
            curr.left = Node(x)
```

```
            return
```

```
        q.append(curr.left)
```

```
        if curr.right is None:
```

```
            curr.right = Node(x)
```

```
            return
```

```
        q.append(curr.right)
```

```
def level_order(root):
```

```
    q = deque([root])
```

```
while q:
    curr = q.popleft()
    print(curr.data, end=" ")
    if curr.left:
        q.append(curr.left)
    if curr.right:
        q.append(curr.right)
```

```
n = int(input())
```

```
if n < 0:
    print("Invalid input")
else:
    arr = list(map(int, input().split()))
    x = int(input())
```

```
root = build_tree(arr)
```

```
if root is None:
    root = Node(x)
else:
    insert(root, x)
```

```
level_order(root)
```

```
#EMPLOYEE REMOVAL MANAGEMENT SYSTEM
```

```
from collections import deque
```

```
class Node:
    def __init__(self, data):
        self.data = data
```

```
self.left = None  
self.right = None
```

```
def build_tree(arr):  
    if not arr or arr[0] == -1:  
        return None
```

```
    root = Node(arr[0])  
    q = deque([root])  
    i = 1
```

```
    while q and i < len(arr):  
        curr = q.popleft()
```

```
        if i < len(arr) and arr[i] != -1:  
            curr.left = Node(arr[i])  
            q.append(curr.left)  
            i += 1
```

```
        if i < len(arr) and arr[i] != -1:  
            curr.right = Node(arr[i])  
            q.append(curr.right)  
            i += 1
```

```
    return root
```

```
def delete_node(root, key):  
    if root is None:  
        return None
```

```
    if root.left is None and root.right is None:
```



```
    if root.data == key:
        return None
    return root

q = deque([root])
key_node = None
last = None
parent = None

while q:
    last = q.popleft()

    if last.data == key:
        key_node = last

    if last.left:
        parent = last
        q.append(last.left)

    if last.right:
        parent = last
        q.append(last.right)

if key_node:
    key_node.data = last.data
    if parent.left == last:
        parent.left = None
    else:
        parent.right = None

return root
```

```

def level_order(root):
    if root is None:
        return
    q = deque([root])
    while q:
        node = q.popleft()
        print(node.data, end=" ")
        if node.left:
            q.append(node.left)
        if node.right:
            q.append(node.right)

n = int(input())

if n < 0:
    print("Invalid input")
else:
    arr = list(map(int, input().split()))
    key = int(input())

    root = build_tree(arr)
    root = delete_node(root, key)
    level_order(root)

#SMART IRRIGATION NETWORK

def postorder(arr, i, n):
    if i >= n:
        return
    postorder(arr, 2 * i + 1, n)
    postorder(arr, 2 * i + 2, n)
    print(arr[i], end=" ")

```

```
n = int(input())
arr = list(map(int, input().split()))
```

```
if len(set(arr)) != n:
    print("Invalid input")
```

```
else:
    postorder(arr, 0, n)
```

```
#HIERARCHICAL STRUCTURE INITIALIZATION SYSTEM
```

```
n = int(input())
```

```
if n < 0:
    print("Invalid input")
```

```
else:
    arr = list(map(int, input().split()))
```

```
if arr[0] == -1:
    print("Empty tree")
```

```
else:
    print(arr[0])
```

```
#EMPLOYEE LOOKUP SYSTEM
```

```
from collections import deque
```

```
n = int(input())
```

```
if n < 0:
    print("Invalid input")
```

```
else:
    arr = list(map(int, input().split()))
    key = int(input())
```

```

q = deque([0])
found = False

while q:
    i = q.popleft()

    if i >= n or arr[i] == -1:
        continue

    if arr[i] == key:
        found = True
        break

    q.append(2 * i + 1)
    q.append(2 * i + 2)

if found:
    print("Found")
else:
    print("Not Found")

#ORGANIZATION SIZE EVALUATION SYSTEM
n = int(input())

if n < 0:
    print("Invalid input")
else:
    arr = list(map(int, input().split()))
    count = 0
    for x in arr:
        if x != -1:
            count += 1

```

```

    print(count)
#SERVER LOAD DISTRIBUTION CHECK
n = int(input())

if n < 1:
    print("Invalid input")
else:
    arr = list(map(int, input().split()))

    level = 0
    index = 0

    while index < n:
        nodes = 2 ** level
        print(sum(arr[index:min(index + nodes, n)]), end=" ")
        index += nodes
        level += 1

```

Module --6

Digital Test Result processor

```

n=int(input())
names=input().split()
answers=input().split()
correct=input().strip()
restricted=set(input().split())
marks=int(input())
if len(names)!=n or len(answers)!=n:
    print("Invalid input")
    exit()

```

```
for ans in answers:
    if len(ans)!=len(correct):
        print("Invalid input")
        exit()
result=[]
for i in range(n):
    if names[i] in restricted:
        continue
    score=0
    for j in range(len(correct)):
        if answers[i][j]==correct[j]:
            score+=marks
    result.append(f"{names[i]}:{score}")
print(" ".join(result))
```

Access premission Checker

```
v=input().strip()
s=input().strip()
if v not in ["True","False"] or s not in ["True","False"]:
    print("Invalid input")
elif v=="True" and s=="True":
    print("Access Granted")
elif v=="False" and s=="False":
    print("Access Denied")
else:
    print("Access Denied")
```

Pet care Record inheritance

```

class animal:
    def __init__(self):
        self.name=input()
class det(animal):
    def an(self):
        self.age=int(input())
        self.breed=input()
        if self.age<=0:
            print("Invalid input")
            exit()
        print(f"Animal Name: {self.name}\nAnimal Age: {self.age}\nDog Breed: {self.breed}" )
a=det()
a.an()
#Academic indentity record system

```

```

class Student:
    def det(self):
        person_name=input()
        student_name=input()
        department=input()
        age=int(input())
        if age<=0:
            print("Invalid input")
        else:
            print(F"Person: {person_name}\nStudent: {student_name}\nDepartment:
{department}\nAge: {age}")
s=Student()
s.det()

```

```

# Library book issue and rturn trackng system

```

```

class Books:

    def __init__(self,name,author):

        self.name=name

        self.author=author

        print(f"Book Issued: {self.name}\nAuthor: {self.author}")

    def __del__(self):

        print(f"Book Returned: {self.name}")

name=input()
author=input()

if name=="" or author=="":

    print("Invalid input")

else:

    b=Books(name,author)

    del b

# Book borrow/Return managment

```

```

class book:

    def __init__(self,title,author):

        self.title=title

        self.author=author

        self.available=True

    def borrow(self):

        if not self.available:

            print("Invalid input")

        else:

            self.available=False

            print("Book borrowed successfully")

    def return_book(self):

        if self.available:

            print("Invalid input")

```



```

        else:

            self.available=True

            print("Book returned successfully")
title=input().strip()
author=input().strip()
if title=="" or author=="":
    print("Invalid input")
else:
    book=book(title,author)
    n=int(input())
    for _ in range(n):
        op=input().strip()
        if op=="borrow":
            book.borrow()
        elif op=="return":
            book.return_book()
        else:
            print("Invalid input")

```

#Square area calculater

```

class Area:
    def __init__(self,side,area):
        self.side=side
        self.area=area
        print("Side:",self.side)
        print("Area:",self.area)
n=int(input())
if n>0:
    side=float(n)
    area=float(n**2)
    f=Area(side,area)

```

```

else:
    print("Invalid input")
#company policy sharing
class company:
    def __init__(self,a,b,c,d):
        self.a=a
        self.b=b
        self.c=c
        self.d=d
        print("Department1:",self.a)
        print("Policy:",self.c)
        print("Department2:",self.b)
        print("Policy:",self.c)
        print("Employees:",self.d)
a=input().strip()
b=input().strip()
c=input().strip()
d=int(input())
if d<=0:
    print("Invalid input")
else:
    x=company(a,b,c,d)
# Flexible billing system
n=list(map(int,input().split()))
if len(n)>5:
    print("Invalid input")
else:
    (print(sum(n)))
#Students performance and ranking
class Student:
    def __init__(self,name,marks):

```

```

        self.name=name

        self.marks=marks

        self.gpa=sum(marks)/3

        self.rank=0
n=int(input())
students=[]
for _ in range(n):
    data=input().split()
    name=data[0]
    marks=list(map(int,data[1:]))
    students.append(Student(name,marks))
for s in students:
    print(f"{s.name} {s.gpa:.2f}")
sorted_students=sorted(students,key=lambda x:x.gpa,reverse=True)
rank=1
for i in range(len(sorted_students)):
    if i>0 and sorted_students[i].gpa<sorted_students[i-1].gpa:
        rank=i+1
    sorted_students[i].rank=rank
for s in students:
    print(f"{s.name} Rank:{s.rank}")

#integrated transport managment
f=input()
b=input()
v=input()
r=int(input())
if r<=0:
    print("Invalid input")
    exit()
print("Vehicle:",v)

```

```
print("Fuel Type:",f)
print("Battery Type:",b)
print("Range:",r)
```

Module----7

```
# Flexible billing system
n=list(map(int,input().split()))
if len(n)>5:
    print("Invalid input")
else:
    (print(sum(n)))
# overriding methods with super
n=int(input())
if n==0:
    print("Invalid input")
    exit()
class logger:
    def dis(self):
        print("General log message")
class filelogger(logger):
    def dis(self):
        super().dis()
        print("File logger writes log")
f=filelogger()
f.dis()
#Notification delivry system
from abc import ABC, abstractmethod
class notification(ABC):
    @abstractmethod
    def send(self):
```

```

        pass

class emailnotification(notification):

    def __init__(self,message):

        self.message=message

    def send(self):

        return self.message


class smsnotification(notification):

    def __init__(self,message):

        self.message=message

    def send(self):

        return self.message

email_message=input()
sms_message=input()
priority_level=int(input())
if priority_level<=0:
    print("Invalid input")
else:
    email=emailnotification(email_message)
    sms=smsnotification(sms_message)
    print(f"Email sent: {email.send()}")
    print(f"SMS sent: {sms.send()}")
    print(f"Priority level is: {priority_level}")

#product discount calaulation

from abc import ABC, abstractmethod

class product(ABC):

    @abstractmethod

    def calculate_discount(self):

        pass

class electronic(product):

    def __init__(self,price,discount):

```

```

        self.price=price

        self.discount=discount

    def calculate_discount(self):

        return self.price-(self.price*self.discount/100)

class clothing(product):

    def __init__(self,price,discount):

        self.price=price

        self.discount=discount

    def calculate_discount(self):

        return self.price-(self.price*self.discount/100)

electronic_price=int(input())
electronic_discount=int(input())
clothing_price=int(input())
clothing_discount=int(input())

if electronic_price<=0 or electronic_discount<=0 or clothing_price<=0 or clothing_discount<=0:

    print("Invalid input")

else:

    electronics=electronic(electronic_price,electronic_discount)

    clothing=clothing(clothing_price,clothing_discount)

    print(f"Electronics final price is: {electronics.calculate_discount():.2f}")

    print(f"Clothing final price is: {clothing.calculate_discount():.2f}")

#Payment method proccesing system

class payment:

    def process(self):

        print("general payment")

    def creditcard(self):

        print("CreditCard payment processed")

    def upi(self):

        print("UPI payment successful")

x=payment()

L=input().strip()

```

```
if L=="CreditCard":
    x.creditcard()
elif L=="UPI":
    x.upi()
else:
    print("Invalid input")
```

#Flexible free claculator

```
def fees(a,b=0):
    if b<a:
        return a-b
    else:
        return "Invalid input"
```

```
a=int(input())
```

```
b=int(input())
```

```
s=fees(a,b)
```

```
print(s)
```

#Speed monitoring

```
from abc import ABC,abstractmethod
```

```
class Vehicle(ABC):
```

```
    @abstractmethod
```

```
    def speed(self,a):
```

```
        pass
```

```
class Car(Vehicle):
```

```
    def speed(self,a):
```

```
        self.a=a
```

```
        print("Car speed is:",self.a)
```

```
class Bike(Vehicle):
```

```
    def speed(self,a):
```

```
        self.a=a
```

```

        print("Bike speed is:",self.a)
c=Car()
b=Bike()
x=int(input())
y=int(input())
if x<=0 or y<=0:
    print("Invalid input")
else:
    c.speed(x)
    b.speed(y)

```

#smart delivery charge system

```

def delivery(base,type):
    if 1<=base<=500:
        if type=="standard":
            return base
        elif type=="express":
            return base+50
        else:
            return "Invalid input"
    else:
        return "Invalid input"
a=int(input())
b=input().strip().lower()
x=delivery(a,b)
print(x)

```

#Authentication system

```

from abc import ABC,abstractmethod
class Auth(ABC):
    @abstractmethod

```



```

def login(self):
    pass
class EmailAuth(Auth):
    def login(self,a):
        self.a=a
        print("EmailAuth login:",self.a)
class OTPAuth(Auth):
    def login(self,b):
        self.b=b
        print("OTPAuth login:",self.b)
a=input().strip()
b=input().strip()
if 1<=len(a)<=100 and 1<=len(b)<=10:
    x=EmailAuth()
    x.login(a)
    y=OTPAuth()
    y.login(int(b))
else:
    print("Invalid input")

```

Module---8

#Numeric input validation system

try:

```

n=input()
s=int(n)
print(s)

```

except ValueError:

```

    print("Invalid input")

```

#Safe message builder system

```

n=input()

```

```
v=input()
try:
    v=int(v)
    result=n+v
    print(result)
except TypeError:
    print("Invalid input")
except ValueError:
    print(n+v)
```

Secure score evaluation system

```
try:
    s=int(input())
except ValueError:
    print("Invalid input")
else:
    if s>=90:
        print("Excellent")
    elif 70<=s<90:
        print("Good")
    elif 40<=s<70:
        print("Average")
    else:
        print("Poor")
```

Excpion relay system

```
try:
    a=int(input())
    b=int(input())
    c=a/b
    print(c)
except (ZeroDivisionError,ValueError):
```

```

    print("Invalid input")
# Age eligiblity verification system
n=int(input())
if n>=18:
    print(n)
else:
    print("Invalid input")
# octal code validation system
s=input()
try:
    if s.isdigit():
        print(int(s,8))
    else:
        print("Invalid input")
except:
    print("Invalid input")
#Safe list aces system
n=int(input())
s=input().split()
i=int(input())
try:
    if i<n:
        print(s[i])
    else:
        print("Invalid input")
except TypeError and IndexError:
    print("Invalid input")
#calender year validation
try:
    Y=int(input())
    if (Y%400==0)or(Y%100!=0 and Y%4==0):

```

```

        print("Leap Year")
    else:
        print("Not a Leap Year")
except:
    print("Invalid input")
#Error bubble up system
try:
    a=int(input())
    b=int(input())
    try:
        c=a/b
        print(int(c)if c.is_integer()else c)
    except ZeroDivisionError:
        raise
except(ValueError,ZeroDivisionError):
    print("Invalid input")
# Secyre record lookup system
n=int(input())
keys=list(map(int,input().split()))
values=list(map(int,input().split()))
d=dict(zip(keys,values))
k=int(input())
try:
    print(d[k])
except KeyError:
    print("Invalid input")

```

Module---9

```

# peak pattern detection
n=int(input())

```

```

if n<3:
    print("Invalid input")
else:
    a=list(map(int,input().split()))
    ans=0
    for i in range(1,n-1):
        if a[i-1]<a[i]>a[i+1]:
            l=r=i
            while l>0 and a[l-1]<a[l]:
                l-=1
            while r<n-1 and a[r]>a[r+1]:
                r+=1
            ans=max(ans,r-l+1)
    print(ans)

```

```

# Matrix boundary reflection
n=int(input())
matrix=[]
for i in range(n):
    matrix.append(list(map(int,input().split())))
result=[row[:]for row in matrix]
for i in range(1,n-1):
    for j in range(1,n-1):
        result[i][j]=matrix[n - 1 - i][n - 1 - j]
for row in result:
    print(*row)

# Spiral layer sum
n=int(input())
matrix=[]
for _ in range(n):
    matrix.append(list(map(int,input().split())))

```

```

top=0
bottom=n-1
left=0
right=n-1
ans=[]
while top<=bottom and left<=right:
    s=0
    for i in range(left,right+1):
        s+=matrix[top][i]
    top+=1
    for i in range(top,bottom+1):
        s+=matrix[i][right]
    right-=1
    if top<=bottom:
        for i in range(right,left-1,-1):
            s+=matrix[bottom][i]
        bottom-=1
    if left<=right:
        for i in range(bottom,top-1,-1):
            s+=matrix[i][left]
        left+=1
    ans.append(s)
print(*ans)

# Sort numbers using bubble sort
n=int(input())
a=list(map(int,input().split()))
if n<=0:
    print("Invalid input")
else:
    for i in range(n):
        for j in range(0,n-i-1):

```

```

        if a[j]>a[j+1]:
            a[j],a[j+1]=a[j+1],a[j]
    print(*a)
    for i in range(n):
        for j in range(0,n-i-1):
            if a[j]<a[j+1]:
                a[j],a[j+1]=a[j+1],a[j]
        print(*a)
# set matrix zeros
m,n=map(int,input().split())
matrix=[]
for i in range(m):
    matrix.append(list(map(int,input().split())))
rows=[]
cols=[]
for i in range(m):
    for j in range(n):
        if matrix[i][j]==0:
            rows.append(i)
            cols.append(j)
for i in range(m):
    for j in range(n):
        if i in rows or j in cols:
            matrix[i][j]=0
for row in matrix:
    print(row)
#Dominat value detection
n=int(input())
if n==0:
    print("Invalid input")
else:

```

```

a=list(map(int,input().split()))
c=a[0]
count=1
for i in range(1,n):
    if a[i]==c:
        count+=1
    else:
        count-=1
        if count==0:
            c=a[i]
            count=1
if a.count(c)>n//2:
    print(c)
else:
    print(-1)
#Water storage estimator
n=int(input())
a=list(map(int,input().split()))
if len(a)!=n:
    print("Invalid input")
else:
    l=[0]*n
    r=[0]*n
    l[0]=a[0]
    for i in range(1,n):
        l[i]=max(l[i-1],a[i])
    r[n-1]=a[n-1]
    for i in range(n-2,-1,-1):
        r[i]=max(r[i+1],a[i])
    ans=0
    for i in range(n):

```



```

        ans+=min(l[i],r[i])-a[i]
    print(ans)
# Count magical rows
n=int(input())
m=int(input())
c=0
for i in range(n):
    a=list(map(int,input().split()))
    s=0
    for j in a:
        if j%2!=0:
            s+=j
    if s%2==0:
        c+=1
print(c)
# The shampoo spill window
n,k=map(int,input().split())
a=list(map(int,input().split()))
if n<0 or k>n or len(a)!=n:
    print("Invalid input")
else:
    s=sum(a[:k])
    m=s
    for i in range(k,n):
        s+=a[i]-a[i-k]
        if s>m:
            m=s
    print(m)
# Balanced tower adjustment
n=int(input())
arr=list(map(int,input().split()))

```

```

k=int(input())
if 0 in arr:
    print("Invalid input")
else:
    arr.sort()
    ans=arr[-1]-arr[0]
    small=arr[0]+k
    big=arr[-1]-k
    if small>big:
        small,big=big,small
    for i in range(n-1):
        mn=min(small,arr[i+1]-k)
        mx=max(big,arr[i]+k)
        if mn<0:
            continue
        ans=min(ans,mx-mn)
    print(ans)

```

Module---10:

#Title case conversion

```
s=input().strip()
```

```
if (not s or any(not(ch.isalpha() or ch.isspace())for ch in s)):
```

```
    print("Invalid input")
```

```
else:
```

```
    print(" ".join (word.title() for word in s.split()))
```

#Rotating a string by position

```
s=input().strip()
```

```
n=int(input())
```

```
if not s or not all (c.isalpha() or c.isspace() for c in s) or n<0 or n>len(s):
```

```
    print("Invalid input")
```

else:

```
print(s[-n:] + s[:-n])
```

Removing Dupilicate characters

```
s=input().strip()
```

```
t=s.replace(" ","")
```

```
r=t.isalpha()
```

```
if r==True:
```

```
    ans=""
```

```
    for i in s:
```

```
        if i not in ans:
```

```
            ans+=i
```

```
    print(ans)
```

```
else:
```

```
    print("Invalid input")
```

Substring occurrence count

```
s=input()
```

```
ser=input()
```

```
t=s.replace(" ","")
```

```
r=t.isalpha()
```

```
if r==True:
```

```
    count=0
```

```
    for i in range(len(s)-len(ser)+1):
```

```
        if s[i:i+len(ser)]==ser:
```

```
            count+=1
```

```
    print(count)
```

```
else:
```

```

    print("Invalid input")
# Secure acces checker
s=input()
upper=False
lower=False
Digit=False
for i in s:
    if i.isupper():
        upper=True
    if i.islower():
        lower=True
    if i.isdigit():
        Digit=True
if upper and lower and Digit and len(s)>=8:
    print("Strong Password")
else:
    print("Invalid input")
# Converting camelcase to snake_case
n=input().strip()
if not n or not n[0].isupper():
    print("Invalid input")
else:
    s=""
    for i,j in enumerate(n):
        if j.isupper():
            if i!=0:
                s=s+"_"
            s=s+j.lower()
        else:
            s=s+j
    print(s)

```

```

#data packet rotation
a=input().strip()
b=input().strip()
if not a.islower() or not b.islower():
    print("Invalid input")
elif len(a)!=len(b):
    print(-1)
else:
    print((a+a).find(b))

#Text analysis repot
n=input()
for i in n:
    if i.isdigit():
        print("Invalid input")
        break
else:
    a=len(n.split())
    b=0
    c=0
    for j in n.lower():
        if j in "aeiou":
            b=b+1
        elif j.isalpha():
            c=c+1
    print(a)
    print(b)
    print(c)

# Palindrome
n=input()
for i in n:

```

```

    if i.isdigit() or (not i.isalpha() and i!=" "):
        print("Invalid input")
        break
else:
    n=n.replace(" ","").lower()
    if n==n[::-1]:
        print("Palindrome")
    else:
        print("Not Palindrome")
#Speech pattern analysis system
n=input().strip()
a=0
for i in n:
    if i in "aeiouAEIOU":
        continue
    elif not i.isalpha():
        continue
    else:
        a=a+1
print(a)

```

BINARY SEARCH TREE:

WORK OUT:

1) # Library shelf arrangement

```

class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

```

```
def insert(root, key):  
    if root is None:  
        return Node(key)  
    if key < root.val:  
        root.left = insert(root.left, key)  
    else:  
        root.right = insert(root.right, key)  
    return root
```

```
def inorder(root, result):  
    if root:  
        inorder(root.left, result)  
        result.append(root.val)  
        inorder(root.right, result)
```

Input

```
try:  
    n = int(input().strip())  
    values = list(map(int, input().split()))  
  
    if len(values) != n or len(set(values)) != n:  
        print("Invalid input")  
    else:  
        root = None  
        for v in values:  
            root = insert(root, v)  
  
        result = []  
        inorder(root, result)  
        print(*result)
```

```
except:
```

```
    print("Invalid input")
```

```
2) # Priority inspection sequence
```

```
class Node:
```

```
    def __init__(self, val):
```

```
        self.val = val
```

```
        self.left = None
```

```
        self.right = None
```

```
def insert(root, key):
```

```
    if root is None:
```

```
        return Node(key)
```

```
    if key < root.val:
```

```
        root.left = insert(root.left, key)
```

```
    elif key > root.val:
```

```
        root.right = insert(root.right, key)
```

```
    return root
```

```
def preorder(root, result):
```

```
    if root:
```

```
        result.append(root.val)
```

```
        preorder(root.left, result)
```

```
        preorder(root.right, result)
```

```
# Input
```

```
try:
```

```
    n = int(input().strip())
```

```
    values = list(map(int, input().split()))
```

```
    if len(values) != n or len(set(values)) != n:
```



```

        print("Invalid input")
    else:
        root = None
        for v in values:
            root = insert(root, v)

        result = []
        preorder(root, result)
        print(*result)

except:
    print("Invalid input")

```

3) # Lost File Locator

```

import sys

class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.val:
        root.left = insert(root.left, key)
    elif key > root.val:
        root.right = insert(root.right, key)
    return root

```

```
def search(root, key):  
    if root is None:  
        return False  
    if root.val == key:  
        return True  
    elif key < root.val:  
        return search(root.left, key)  
    else:  
        return search(root.right, key)
```

Input

```
try:  
    n_str = sys.stdin.readline().strip()  
    if not n_str.isdigit():  
        sys.stdout.write("Invalid input\n")  
        sys.exit()  
    n = int(n_str)  
  
    if n < 1 or n > 10:  
        sys.stdout.write("Invalid input\n")  
        sys.exit()  
  
    values_line = sys.stdin.readline().strip()  
    values_str = values_line.split()  
  
    if len(values_str) != n:  
        sys.stdout.write("Invalid input\n")  
        sys.exit()  
  
    values = []
```

```

seen_values = set()
for val_str in values_str:
    if not val_str.isdigit(): # Constraints 10 <= value <= 100 imply positive integers
        sys.stdout.write("Invalid input\n")
        sys.exit()
    v = int(val_str)
    if v < 10 or v > 100:
        sys.stdout.write("Invalid input\n")
        sys.exit()
    if v in seen_values:
        sys.stdout.write("Invalid input\n")
        sys.exit()
    seen_values.add(v)
    values.append(v)

```

```

k_str = sys.stdin.readline().strip()
# K can be any integer, so check for negative sign as well
if not (k_str.isdigit() or (k_str.startswith('-') and k_str[1:].isdigit())):
    sys.stdout.write("Invalid input\n")
    sys.exit()
k = int(k_str)

```

```

# Build BST
root = None
for v in values:
    root = insert(root, v)

```

```

# Search
if search(root, k):
    sys.stdout.write("Found\n")
else:

```

```
sys.stdout.write("Not Found\n")
```

```
except EOFError:
```

```
    sys.stdout.write("Invalid input\n")
```

```
except ValueError:
```

```
    sys.stdout.write("Invalid input\n")
```

```
except Exception:
```

```
    sys.stdout.write("Invalid input\n")
```

4) # Structure rule verification

```
class Node:
```

```
    def __init__(self, val):
```

```
        self.val = val
```

```
        self.left = None
```

```
        self.right = None
```

```
def build_tree(values):
```

```
    if not values or values[0] == -1:
```

```
        return None
```

```
    nodes = []
```

```
    for v in values:
```

```
        node = None if v == -1 else Node(v)
```

```
        nodes.append(node)
```

```
    i = 0
```

```
    for j in range(1, len(values), 2):
```

```
        if nodes[i]:
```

```
            if j < len(values):
```

```
                nodes[i].left = nodes[j]
```

```
            if j + 1 < len(values):
```

```

        nodes[i].right = nodes[j + 1]

    i += 1

return nodes[0]

def is_bst(root, low=float('-inf'), high=float('inf')):
    if root is None:
        return True
    if not (low < root.val < high):
        return False
    return (is_bst(root.left, low, root.val) and
            is_bst(root.right, root.val, high))

# Input
try:
    n = int(input().strip())
    values = list(map(int, input().split()))

    if len(values) != n or values[0] == -1:
        print("Invalid input")
    else:
        root = build_tree(values)

        if is_bst(root):
            print("Valid BST")
        else:
            print("Invalid BST")

except:
    print("Invalid input")

```

5) # Warehouse item Removal

```
class Node:
```

```
    def __init__(self, val):
```

```
        self.val = val
```

```
        self.left = None
```

```
        self.right = None
```

```
def insert(root, key):
```

```
    if root is None:
```

```
        return Node(key)
```

```
    if key < root.val:
```

```
        root.left = insert(root.left, key)
```

```
    elif key > root.val:
```

```
        root.right = insert(root.right, key)
```

```
    return root
```

```
def minValueNode(node):
```

```
    current = node
```

```
    while current.left:
```

```
        current = current.left
```

```
    return current
```

```
def deleteNode(root, key):
```

```
    if root is None:
```

```
        return root
```

```
    if key < root.val:
```

```
        root.left = deleteNode(root.left, key)
```

```
    elif key > root.val:
```

```
        root.right = deleteNode(root.right, key)
```

```
    else:
```

```

if root.left is None:
    return root.right
elif root.right is None:
    return root.left

temp = minValueNode(root.right)
root.val = temp.val
root.right = deleteNode(root.right, temp.val)

```

```

return root

```

```

def inorder(root, res):
    if root:
        inorder(root.left, res)
        res.append(root.val)
        inorder(root.right, res)

```

```

# Input

```

```

try:
    n = int(input().strip())
    values = list(map(int, input().split()))
    k = int(input().strip())

    if len(values) != n or len(set(values)) != n or k not in values:
        print("Invalid input")
    else:
        root = None
        for v in values:
            root = insert(root, v)

        root = deleteNode(root, k)

```

```
res = []  
inorder(root, res)  
print(*res)
```

except:

```
print("Invalid input")
```

PRACTICE:

1) # SORTED record display

class Node:

```
def __init__(self, val):  
    self.val = val  
    self.left = None  
    self.right = None
```

def insert(root, key):

```
    if root is None:  
        return Node(key)  
    if key < root.val:  
        root.left = insert(root.left, key)  
    elif key > root.val:  
        root.right = insert(root.right, key)  
    return root
```

def inorder(root, result):

```
    if root:  
        inorder(root.left, result)  
        result.append(root.val)  
        inorder(root.right, result)
```



```

try:

    n = int(input().strip())
    values = list(map(int, input().split()))

    if len(values) != n or n > 10:
        print("Invalid input")
    else:
        root = None
        for v in values:
            root = insert(root, v)

        result = []
        inorder(root, result)
        print(*result)

except:
    print("Invalid input")

```

2) # Final processing sequence

```

class Node:

    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.val:
        root.left = insert(root.left, key)
    elif key > root.val:

```

```
    root.right = insert(root.right, key)
return root
```

```
def postorder(root, result):
    if root:
        postorder(root.left, result)
        postorder(root.right, result)
        result.append(root.val)
```

```
# Input
```

```
try:
    n = int(input().strip())
    values = list(map(int, input().split()))

    if len(values) != n or any(v < 10 or v > 100 for v in values):
        print("Invalid input")
    else:
        root = None
        for v in values:
            root = insert(root, v)

        result = []
        postorder(root, result)
        print(*result)

except:
    print("Invalid input")
```

```
3) # Floor-wise inspection report
from collections import deque
```

```
class Node:

    def __init__(self, val):

        self.val = val

        self.left = None

        self.right = None


def insert(root, key):

    if root is None:

        return Node(key)

    if key < root.val:

        root.left = insert(root.left, key)

    elif key > root.val:

        root.right = insert(root.right, key)

    return root


def level_order(root):

    if not root:

        return []

    result = []

    q = deque([root])

    while q:

        node = q.popleft()

        result.append(node.val)

        if node.left:

            q.append(node.left)

        if node.right:

            q.append(node.right)
```

```
return result
```

```
# Input
```

```
try:
```

```
    n = int(input().strip())
```

```
    values = list(map(int, input().split()))
```

```
    if len(values) != n:
```

```
        print("Invalid input")
```

```
    else:
```

```
        root = None
```

```
        for v in values:
```

```
            root = insert(root, v)
```

```
        result = level_order(root)
```

```
        print(*result)
```

```
except:
```

```
    print("Invalid input")
```

```
4) # Boundary value detector
```

```
class Node:
```

```
    def __init__(self, val):
```

```
        self.val = val
```

```
        self.left = None
```

```
        self.right = None
```

```
def insert(root, key):
```

```
    if root is None:
```

```
        return Node(key)
```

```
    if key < root.val:
```

```

        root.left = insert(root.left, key)
    elif key > root.val:
        root.right = insert(root.right, key)
    return root

def find_min(root):
    while root.left:
        root = root.left
    return root.val

def find_max(root):
    while root.right:
        root = root.right
    return root.val

# Input
try:
    n = int(input().strip())
    values = list(map(int, input().split()))

    if len(set(values)) == 1:
        print("Invalid input")
    else:
        root = None
        for v in values:
            root = insert(root, v)

        minimum = find_min(root)
        maximum = find_max(root)

        print(minimum, maximum)

```

except:

```
    print("Invalid input")
```

5) # Ranked value retrival

class Node:

```
    def __init__(self, val):
```

```
        self.val = val
```

```
        self.left = None
```

```
        self.right = None
```

```
def insert(root, key):
```

```
    if root is None:
```

```
        return Node(key)
```

```
    if key < root.val:
```

```
        root.left = insert(root.left, key)
```

```
    elif key > root.val:
```

```
        root.right = insert(root.right, key)
```

```
    return root
```

```
def inorder(root, res):
```

```
    if root:
```

```
        inorder(root.left, res)
```

```
        res.append(root.val)
```

```
        inorder(root.right, res)
```

```
try:
```

```
    n = int(input().strip())
```

```
    values_str = input().split()
```

```
    values = []
```

```
    for val_str in values_str:
```

```
        values.append(int(val_str))
```

```

k = int(input().strip())

is_valid = True
if not (1 <= n <= 10 and 1 <= k <= 10 and len(values) == n and k <= n):
    is_valid = False
else:
    for v in values:
        if not (10 <= v <= 100):
            is_valid = False
            break

if not is_valid:
    result = "Invalid input"
else:
    root = None
    for v in values:
        root = insert(root, v)

    res = []
    inorder(root, res)
    result = str(res[k-1])

except (ValueError, IndexError):
    result = "Invalid input"
print(result)

```

GRAPHS 1

WORKOUT:

1) # City Traffic map

```
n = int(input())
```

```
if n < 1 or n > 10:
```

```
    print("Invalid input")
```

```
    exit()
```

```
m = int(input())
```

```
if m < 0 or m > 100:
```

```
    print("Invalid input")
```

```
    exit()
```

```
matrix = [[0]*n for _ in range(n)]
```

```
for _ in range(m):
```

```
    u, v = map(int, input().split())
```

```
    if u < 0 or u >= n or v < 0 or v >= n:
```

```
        print("Invalid input")
```

```
        exit()
```

```
    matrix[u][v] = 1
```

```
    matrix[v][u] = 1
```

```
for i in range(n):
```

```
    for j in range(n):
```

```
        if j == n - 1:
```

```
            print(matrix[i][j], end="")
```

```
        else:
```

```
            print(matrix[i][j], end=" ")
```

```
    print()
```

```
2) # social network mapping
```



```
n = int(input())

if n < 0:
    print("Invalid input")

elif n == 0:
    print("Empty network")

else:
    m = int(input())

    if m < 0 or m > (n * (n - 1)) // 2:
        print("Invalid input")
    else:
        adj_list = {i: [] for i in range(n)}
        valid = True

        for _ in range(m):
            u, v = map(int, input().split())

            if u < 0 or u >= n or v < 0 or v >= n:
                valid = False
            else:
                adj_list[u].append(v)
                adj_list[v].append(u)

        if not valid:
            print("Invalid input")
        else:
            for node in range(n):
                print(f"{node}: ", ' '.join(map(str, adj_list[node])))
```

3) # Remove offline server

```
def is_valid(n, m, edges, s):
```

```
    if n < 0 or m < 0:
```

```
        return False
```

```
    for u, v in edges:
```

```
        if u < 0 or u >= n or v < 0 or v >= n:
```

```
            return False
```

```
    if s < 0 or s >= n:
```

```
        return False
```

```
    return True
```

```
def remove_server(edges, s):
```

```
    updated_edges = []
```

```
    for u, v in edges:
```

```
        if u != s and v != s:
```

```
            updated_edges.append((u, v))
```

```
    return updated_edges
```

```
def print_edges(edges):
```

```
    for u, v in edges:
```

```
        print(f"{u} {v}")
```

```
n = int(input())
```

```
if n == 0:
```

```
    print("Empty network")
```

```
elif n < 0:
```

```
    print("Invalid input")
```

```
else:
```

```
m = int(input())
```

```
if m < 0:
```

```
    print("Invalid input")
```

```
else:
```

```
    edges = []
```

```
    for _ in range(m):
```

```
        u, v = map(int, input().split())
```

```
        edges.append((u, v))
```

```
s = int(input())
```

```
if not is_valid(n, m, edges, s):
```

```
    print("Invalid input")
```

```
else:
```

```
    updated_edges = remove_server(edges, s)
```

```
    print_edges(updated_edges)
```

```
4) # Fire evacuation simulation
```

```
from collections import deque
```

```
first_line = input().split()
```

```
if len(first_line) != 2:
```

```
    print("Invalid input")
```

```
else:
```

```
    N, M = int(first_line[0]), int(first_line[1])
```

```
if N <= 0:
```

```
    print("Invalid input")
```

```
else:
```

```
adj = [[] for _ in range(N)]
```

```
valid = True
```

```
for _ in range(M):
```

```
    line = input().split()
```

```
    if len(line) != 2:
```

```
        valid = False
```

```
        break
```

```
    u, v = int(line[0]), int(line[1])
```

```
    if u < 0 or v < 0 or u >= N or v >= N:
```

```
        valid = False
```

```
        break
```

```
    adj[u].append(v)
```

```
    adj[v].append(u)
```

```
if not valid:
```

```
    print("Invalid input")
```

```
else:
```

```
    S_line = input().split()
```

```
    if len(S_line) != 1:
```

```
        print("Invalid input")
```

```
    else:
```

```
        S = int(S_line[0])
```

```
        if S < 0 or S >= N:
```

```
            print("Invalid input")
```

```
        else:
```

```
            visited = [False] * N
```

```
            queue = deque([S])
```

```
            visited[S] = True
```

```
            bfs_order = []
```

```

while queue:
    node = queue.popleft()
    bfs_order.append(node)
    for neighbor in sorted(adj[node]): # sorted for consistent order
        if not visited[neighbor]:
            visited[neighbor] = True
            queue.append(neighbor)

print(*bfs_order)

```

5) # Pipeline circuit verification

```
from collections import deque
```

```
try:
```

```
    N, M = map(int, input().split())
```

```
except:
```

```
    print("Invalid input")
```

```
    exit()
```

```
if N <= 0 or N > 150 or M <= 0 or M > 1500:
```

```
    print("Invalid input")
```

```
    exit()
```

```
adj = [[] for _ in range(N)]
```

```
valid = True
```

```
for _ in range(M):
```

```
    line = input().split()
```

```
    if len(line) != 2:
```

```
        valid = False
```

```

        break
    try:
        u, v = int(line[0]), int(line[1])
    except:
        valid = False
        break
    if u < 0 or v < 0 or u >= N or v >= N:
        valid = False
        break
    adj[u].append(v)
    adj[v].append(u)

if not valid:
    print("Invalid input")
    exit()

visited = [False] * N
has_cycle = False

for i in range(N):
    if not visited[i]:
        stack = [(i, -1)]
        while stack:
            node, parent = stack.pop()
            if not visited[node]:
                visited[node] = True
                for neighbor in adj[node]:
                    if not visited[neighbor]:
                        stack.append((neighbor, node))
                    elif neighbor != parent:
                        has_cycle = True

```

```

        if has_cycle:
            break
    if has_cycle:
        break

print("Yes" if has_cycle else "No")

practice:

1) # Flight route grid
n = int(input())

if n < 0:
    print("Invalid input")

elif n == 0:
    print("Empty graph")

else:
    m = int(input())

    if m < 0 or m > (n * (n - 1)) // 2:
        print("Invalid input")
    else:
        adj_matrix = [[0 for _ in range(n)] for _ in range(n)]
        valid = True

        for _ in range(m):
            u, v = map(int, input().split())

            if u < 0 or u >= n or v < 0 or v >= n:
                valid = False

```

```

    else:

        adj_matrix[u][v] = 1

        adj_matrix[v][u] = 1

    if not valid:

        print("Invalid input")

    else:

        for row in adj_matrix:

            print(' '.join(map(str, row)))

2) # Power grid connection

n = int(input())

if n < 0:

    print("Invalid input")

elif n == 0:

    print("Empty grid")

else:

    m = int(input())

    if m < 0 or m > (n * (n - 1)) // 2:

        print("Invalid input")

    else:

        adj_list = {i: [] for i in range(n)}

        valid = True

        for _ in range(m):

            u, v = map(int, input().split())

            if u < 0 or u >= n or v < 0 or v >= n:

```



```

        valid = False

    else:

        adj_list[u].append(v)
        adj_list[v].append(u)

    if not valid:

        print("Invalid input")

    else:

        for node in range(n):

            print(f"{node}: ", ' '.join(map(str, adj_list[node])))

3) # Remove absolute module

def is_valid(n, m, edges, mod):

    if n < 0 or m < 0:

        return False

    for u, v in edges:

        if u < 0 or u >= n or v < 0 or v >= n:

            return False

    if mod < 0 or mod >= n:

        return False

    return True

def remove_module(edges, mod):

    updated_edges = []

    for u, v in edges:

        if u != mod and v != mod:

            updated_edges.append((u, v))

    return updated_edges

def print_dependencies(edges):

    for u, v in edges:

        print(f"{u} {v}")

```

```

n = int(input())

if n == 0:
    print("Empty system")
elif n < 0:
    print("Invalid input")
else:
    m = int(input())
    if m < 0:
        print("Invalid input")
    else:
        edges = []
        for _ in range(m):
            u, v = map(int, input().split())
            edges.append((u, v))

        mod = int(input())

        if not is_valid(n, m, edges, mod):
            print("Invalid input")
        else:
            updated_edges = remove_module(edges, mod)
            print_dependencies(updated_edges)

```

4) # Dungeon exploration route

```

from collections import deque

```

```

first_line = input().split()

```

```

if len(first_line) != 2:

```

```

    print("Invalid input")
else:
    N, M = int(first_line[0]), int(first_line[1])

    if N <= 0 or N > 200 or M < 0 or M > 2000:
        print("Invalid input")
    else:
        adj = [[] for _ in range(N)]
        valid = True

        for _ in range(M):
            line = input().split()
            if len(line) != 2:
                valid = False
                break
            u, v = int(line[0]), int(line[1])
            if u < 0 or v < 0 or u >= N or v >= N:
                valid = False
                break
            adj[u].append(v)
            adj[v].append(u)

        if not valid:
            print("Invalid input")
        else:
            S_line = input().split()
            if len(S_line) != 1:
                print("Invalid input")
            else:
                S = int(S_line[0])
                if S < 0 or S >= N:

```

```

        print("Invalid input")
    else:
        visited = [False] * N
        queue = deque([S])
        visited[S] = True
        bfs_order = []

        while queue:
            node = queue.popleft()
            bfs_order.append(node)
            for neighbor in sorted(adj[node]):
                if not visited[neighbor]:
                    visited[neighbor] = True
                    queue.append(neighbor)

        print(*bfs_order)

5) # Robot patrol path check
from collections import deque

try:
    N, M = map(int, input().split())
except:
    print("Invalid input")
    exit()

if N <= 0 or N > 200 or M < 0 or M > 2000:
    print("Invalid input")
    exit()

adj = [[] for _ in range(N)]
valid = True

```

```

for _ in range(M):
    line = input().split()
    if len(line) != 2:
        valid = False
        break
    try:
        u, v = int(line[0]), int(line[1])
    except:
        valid = False
        break
    if u < 0 or u >= N or v < 0 or v >= N:
        valid = False
        break
    adj[u].append(v)
    adj[v].append(u)

if not valid:
    print("Invalid input")
    exit()

visited = [False] * N
has_cycle = False

for i in range(N):
    if not visited[i]:
        stack = [(i, -1)]
        while stack:
            node, parent = stack.pop()
            if not visited[node]:
                visited[node] = True

```

```

        for neighbor in adj[node]:
            if not visited[neighbor]:
                stack.append((neighbor, node))
            elif neighbor != parent:
                has_cycle = True
        if has_cycle:
            break
    if has_cycle:
        break

print("Yes" if has_cycle else "No")

```

GRAPHS 2

WORKOUT:

1) # connected graph detection

```

def dfs(node, graph, visited, n):
    visited[node] = True
    for i in range(n):
        if graph[node][i] == 1 and not visited[i]:
            dfs(i, graph, visited, n)

```

```

n = int(input())

```

```

if n < 1 or n > 20:
    print("Invalid input")
    exit()

```

```

graph = []

```

```

for i in range(n):
    row = list(map(int, input().split()))

```

```

if len(row) != n:
    print("Invalid input")
    exit()

for val in row:
    if val < 0 or val > 1:
        print("Invalid input")
        exit()

graph.append(row)

visited = [False] * n

dfs(0, graph, visited, n)

for v in visited:
    if not v:
        print("Graph is Not Connected")
        exit()

print("Graph is Connected")

2) # Minimum transfer cost between libraries
import heapq

def main():
    N, M = map(int, input().split())

    library_to_idx = {}
    idx_to_library = []

```

```
for i in range(N):
    code = input().strip()
    library_to_idx[code] = i
    idx_to_library.append(code)

graph = [[] for _ in range(N)]
invalid = False

for _ in range(M):
    A, B, W = input().split()
    W = int(W)

    if A not in library_to_idx or B not in library_to_idx:
        invalid = True

    u = library_to_idx.get(A, -1)
    v = library_to_idx.get(B, -1)

    if u != -1 and v != -1:
        graph[u].append((v, W))
        graph[v].append((u, W))

S, D = input().split()

if S not in library_to_idx or D not in library_to_idx:
    invalid = True

if invalid:
    print("Invalid input")
    return
```



```

dist = [float('inf')] * N
dist[library_to_idx[S]] = 0
pq = [(0, library_to_idx[S])]

while pq:
    cost, u = heapq.heappop(pq)

    if cost > dist[u]:
        continue

    for v, w in graph[u]:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            heapq.heappush(pq, (dist[v], v))

result = dist[library_to_idx[D]]

if result == float('inf'):
    print("No path")
else:
    print(result)

if __name__ == "__main__":
    main()

3) # Workflow route analysis
first_line = input().split()
if len(first_line) != 2 or not first_line[0].isdigit() or not first_line[1].isdigit():
    print("Invalid input")
    exit()

```

```
N, M = int(first_line[0]), int(first_line[1])
```

```
if N < 1 or N > 20 or M < 0 or M > 150:
```

```
    print("Invalid input")
```

```
    exit()
```

```
adj = [[] for _ in range(N)]
```

```
valid = True
```

```
for _ in range(M):
```

```
    line = input().split()
```

```
    if len(line) != 2 or not line[0].isdigit() or not line[1].isdigit():
```

```
        valid = False
```

```
        break
```

```
    u, v = int(line[0]), int(line[1])
```

```
    if u < 0 or u >= N or v < 0 or v >= N:
```

```
        valid = False
```

```
        break
```

```
    adj[u].append(v)
```

```
last_line = input().split()
```

```
if len(last_line) != 2 or not last_line[0].isdigit() or not last_line[1].isdigit():
```

```
    valid = False
```

```
else:
```

```
    X, Y = int(last_line[0]), int(last_line[1])
```

```
    if X < 0 or X >= N or Y < 0 or Y >= N or X == Y:
```

```
        valid = False
```

```
if not valid:
```

```
    print("Invalid input")
```

```
exit()
```

```
paths = []
```

```
def dfs(node, path, visited):
```

```
    visited[node] = True
```

```
    path.append(node)
```

```
    if node == Y:
```

```
        paths.append(path[:])
```

```
    else:
```

```
        for neighbor in adj[node]:
```

```
            if not visited[neighbor]:
```

```
                dfs(neighbor, path, visited)
```

```
    path.pop()
```

```
    visited[node] = False
```

```
visited = [False] * N
```

```
dfs(X, [], visited)
```

```
if len(paths) == 0:
```

```
    print("No path exists")
```

```
else:
```

```
    for p in paths:
```

```
        print(" ".join(map(str, p)))
```

```
4) # Maximum production timeline in scene workflow
```

```
from collections import defaultdict, deque
```

```
def find_longest_path_dag(n, m, scenes, dependencies, start, end):
```

```
if start not in scenes or end not in scenes:
```

```
    return "Invalid input"
```

```
graph = defaultdict(list)
```

```
in_degree = {scene: 0 for scene in scenes}
```

```
for u, v, w in dependencies:
```

```
    graph[u].append((v, w))
```

```
    in_degree[v] += 1
```

```
topo_order = []
```

```
queue = deque([s for s in scenes if in_degree[s] == 0])
```

```
while queue:
```

```
    u = queue.popleft()
```

```
    topo_order.append(u)
```

```
    for v, _ in graph[u]:
```

```
        in_degree[v] -= 1
```

```
        if in_degree[v] == 0:
```

```
            queue.append(v)
```

```
dist = {scene: float('-inf') for scene in scenes}
```

```
dist[start] = 0
```

```
for u in topo_order:
```

```
    if dist[u] != float('-inf'):
```

```
        for v, w in graph[u]:
```

```
            if dist[v] < dist[u] + w:
```

```
                dist[v] = dist[u] + w
```

```
return dist[end] if dist[end] != float('-inf') else "No path"
```

```

def main():
    try:
        N, M = map(int, input().split())
        scenes = [input().strip() for _ in range(N)]
        dependencies = []
        for _ in range(M):
            parts = input().split()
            dependencies.append((parts[0], parts[1], int(parts[2])))
        S, D = input().split()
        result = find_longest_path_dag(N, M, scenes, dependencies, S, D)
        print(result)
    except Exception:
        print("Invalid input")

```

```

if __name__ == "__main__":

```

```

    main()

```

5) # Friendship link vulnerability

```

def bridge_dfs(u, parent, visited, disc, low, graph, time, bridges, idx_to_name):

```

```

    visited[u] = True

```

```

    disc[u] = low[u] = time[0]

```

```

    time[0] += 1

```

```

    for v in graph[u]:

```

```

        if not visited[v]:

```

```

            bridge_dfs(v, u, visited, disc, low, graph, time, bridges, idx_to_name)

```

```

            low[u] = min(low[u], low[v])

```

```

            if low[v] > disc[u]:

```

```

                name_u = idx_to_name[u]

```

```

                name_v = idx_to_name[v]

```

```

                bridges.append(tuple(sorted([name_u, name_v])))

```

```
elif v != parent:
```

```
    low[u] = min(low[u], disc[v])
```

```
def main():
```

```
    N, M = map(int, input().split())
```

```
    if N < 1 or N > 100 or M < 0 or M > 1000:
```

```
        print("Invalid input")
```

```
        return
```

```
    member_to_idx = {}
```

```
    idx_to_name = []
```

```
    invalid = False
```

```
    for i in range(N):
```

```
        name = input().strip()
```

```
        if any(c.islower() for c in name):
```

```
            invalid = True
```

```
        if name in member_to_idx:
```

```
            invalid = True
```

```
        member_to_idx[name] = i
```

```
        idx_to_name.append(name)
```

```
    graph = [[] for _ in range(N)]
```

```
    edges_set = set()
```

```
    for _ in range(M):
```

```
        A, B = input().split()
```

```
        if A not in member_to_idx or B not in member_to_idx:
```

```
            invalid = True
```

```
            break
```

```
    u, v = member_to_idx[A], member_to_idx[B]
```

```

    if (min(u,v), max(u,v)) in edges_set:
        continue

    edges_set.add((min(u,v), max(u,v)))

    graph[u].append(v)
    graph[v].append(u)

if invalid:
    print("Invalid input")
    return

visited = [False] * N
disc = [0] * N
low = [0] * N
time = [0]
bridges = []

for i in range(N):
    if not visited[i]:
        bridge_dfs(i, -1, visited, disc, low, graph, time, bridges, idx_to_name)

if not bridges:
    print("No bridges")
else:
    bridges.sort()
    for a, b in bridges:
        print(a, b)

if __name__ == "__main__":
    main()

```

PRACTECE

```
1) # Symmentry citation map creation
```

```
import sys
```

```
from collections import defaultdict
```

```
def convert_citations(N, M, edges):
```

```
    if N <= 0:
```

```
        return "Invalid input"
```

```
    adj = defaultdict(set)
```

```
    for u, v in edges:
```

```
        adj[u].add(v)
```

```
        adj[v].add(u)
```

```
    result = []
```

```
    for node in range(N):
```

```
        neighbors = sorted(list(adj[node]))
```

```
        line = f"{node}: {' '.join(map(str, neighbors))}" if neighbors else f"{node}:"
```

```
        result.append(line)
```

```
    return result
```

```
def main():
```

```
    lines = sys.stdin.read().splitlines()
```

```
    N, M = map(int, lines[0].split())
```

```
    edges = [tuple(map(int, line.split())) for line in lines[1:1+M]]
```

```
    output = convert_citations(N, M, edges)
```

```
    if isinstance(output, str):
```

```
        print(output)
```

```
    else:
```

```
        for line in output:
```

```
            print(line)
```

```
if __name__ == "__main__":
```

```
    main()
```


2) # Material transfer efficiency

```
import heapq
```

```
first_line = input().split()
```

```
if len(first_line) != 2 or not first_line[0].isdigit() or not first_line[1].isdigit():
```

```
    print("Invalid input")
```

```
    exit()
```

```
N, M = int(first_line[0]), int(first_line[1])
```

```
if N < 1 or N > 100 or M < 0 or M > 1000:
```

```
    print("Invalid input")
```

```
    exit()
```

```
adj = [[] for _ in range(N)]
```

```
valid = True
```

```
for _ in range(M):
```

```
    line = input().split()
```

```
    if len(line) != 3 or not all(x.isdigit() for x in line):
```

```
        valid = False
```

```
        break
```

```
u, v, w = map(int, line)
```

```
if u < 0 or u >= N or v < 0 or v >= N or w < 0 or w > 100:
```

```
    valid = False
```

```
    break
```

```
if u == v:
```

```
    valid = False
```

```
break
```

```
adj[u].append((v, w))
```

```
adj[v].append((u, w))
```

```
last_line = input().split()
```

```
if len(last_line) != 2 or not all(x.isdigit() for x in last_line):
```

```
    valid = False
```

```
else:
```

```
    X, Y = map(int, last_line)
```

```
    if X < 0 or X >= N or Y < 0 or Y >= N:
```

```
        valid = False
```

```
if not valid:
```

```
    print("Invalid input")
```

```
    exit()
```

```
dist = [float('inf')] * N
```

```
dist[X] = 0
```

```
heap = [(0, X)]
```

```
while heap:
```

```
    d, node = heapq.heappop(heap)
```

```
    if d > dist[node]:
```

```
        continue
```

```
    for neighbor, w in adj[node]:
```

```
        if dist[node] + w < dist[neighbor]:
```

```
            dist[neighbor] = dist[node] + w
```

```
            heapq.heappush(heap, (dist[neighbor], neighbor))
```

```
if dist[Y] == float('inf'):
```

```

    print("Transfer impossible")
else:
    print(dist[Y])

3) # Maximum learning path in course dependency DAG

from collections import deque

def main():
    N, M = map(int, input().split())

    graph = [[] for _ in range(N)]
    indegree = [0] * N

    for _ in range(M):
        u, v, w = map(int, input().split())
        graph[u].append((v, w))
        indegree[v] += 1

    S, D = map(int, input().split())

    queue = deque()
    topo = []

    for i in range(N):
        if indegree[i] == 0:
            queue.append(i)

    while queue:
        node = queue.popleft()
        topo.append(node)

        for nei, _ in graph[node]:

```

```

        indegree[nei] -= 1

    if indegree[nei] == 0:
        queue.append(nei)

if len(topo) != N:
    print("Invalid input")
    return

dist = [-float('inf')] * N
dist[S] = 0

for u in topo:
    if dist[u] != -float('inf'):
        for v, w in graph[u]:
            if dist[u] + w > dist[v]:
                dist[v] = dist[u] + w

if dist[D] == -float('inf'):
    print("No path")
else:
    print(dist[D])

if __name__ == "__main__":
    main()

4) # Critical tree connected detection in forest
def bridge_dfs(u, parent, visited, disc, low, graph, time, bridges):
    visited[u] = True
    disc[u] = low[u] = time[0]
    time[0] += 1

    for v in graph[u]:

```

```

    if not visited[v]:
        bridge_dfs(v, u, visited, disc, low, graph, time, bridges)
        low[u] = min(low[u], low[v])
        if low[v] > disc[u]:
            bridges.append((min(u, v), max(u, v)))
    elif v != parent:
        low[u] = min(low[u], disc[v])

def main():
    N, M = map(int, input().split())

    if N < 1 or N > 100 or M < 0 or M > 1000:
        print("Invalid input")
        return

    edges_set = set()
    graph = [[] for _ in range(N)]
    invalid = False

    for _ in range(M):
        u, v = map(int, input().split())
        if u < 0 or u >= N or v < 0 or v >= N:
            invalid = True
        if (min(u, v), max(u, v)) in edges_set:
            invalid = True
        edges_set.add((min(u, v), max(u, v)))
        graph[u].append(v)
        graph[v].append(u)

    if invalid:
        print("Invalid input")

```

```

    return

visited = [False] * N
disc = [0] * N
low = [0] * N
time = [0]
bridges = []

for i in range(N):
    if not visited[i]:
        bridge_dfs(i, -1, visited, disc, low, graph, time, bridges)

if not bridges:
    print("No bridges")
else:
    bridges.sort()
    for u, v in bridges:
        print(u, v)

if __name__ == "__main__":
    main()

```

5) # Prerequisite chain validation

```

first_line = input().split()
if len(first_line) != 2 or not all(x.isdigit() for x in first_line):
    print("Invalid input")
    exit()

```

```

N, M = map(int, first_line)

```

```

if not (1 <= N <= 100) or not (0 <= M <= 500):
    print("Invalid input")
    exit()

```

```
adj = [[] for _ in range(N)]
```

```
for _ in range(M):
```

```
    line = input().split()
```

```
    if len(line) != 2 or not all(x.isdigit() for x in line):
```

```
        print("Invalid input")
```

```
        exit()
```

```
    u, v = map(int, line)
```

```
    if not (0 <= u < N) or not (0 <= v < N):
```

```
        print("Invalid input")
```

```
        exit()
```

```
    adj[u].append(v)
```

```
def has_cycle():
```

```
    visited = [False]*N
```

```
    rec_stack = [False]*N
```

```
def dfs(node):
```

```
    visited[node] = True
```

```
    rec_stack[node] = True
```

```
    for neighbor in adj[node]:
```

```
        if not visited[neighbor]:
```

```
            if dfs(neighbor):
```

```
                return True
```

```
        elif rec_stack[neighbor]:
```

```
            return True
```

```
    rec_stack[node] = False
```

```
    return False
```

```

for i in range(N):
    if not visited[i]:
        if dfs(i):
            return True
return False

```

```

if has_cycle():
    print("Circular prerequisite found")
else:
    print("No circular prerequisite")

```

INDEX SEARCHING & SORTING

WORKOUT:

1) # Sort numbers using bubble sort

```

n=int(input())
if n<=0:
    print("Invalid input")
else:
    arr = list(map(int,input().split()))
    asc = arr[:]
    for i in range(n):
        for j in range(0,n-i-1):
            if asc[j]>asc[j+1]:
                asc[j],asc[j+1]=asc[j+1],asc[j]
    print(*asc)
    desc = asc[::-1]
    print(*desc)

```

2) # Product price arrangement


```

n=int(input())
arr=list(map(int,input().split()))
if len(set(arr)) != n:
    print("Invalid Input")
else:
    for i in range(1, n):
        key = arr[i]
        j=i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1]= arr[j]
            j -= 1
        arr[j + 1] = key
    print(*arr)
3) # Binary search-satellite frequency lookup

```

```

n= int(input())
arr = list(map(int,input().split()))
k = int(input())
for i in range(n-1):
    if arr[i] >arr[i+1]:
        print("Invalid Input")
        exit()
low=0
high=n-1
while low<=high:
    mid =(low+high)//2
    if arr [mid]==k:
        print(mid+1)
        break
    elif arr[mid]<k:
        low = mid+1
else:

```

```

        high=mid-1
else:
    print(-1)

4) # merge overlapping time slots
n=int(input())
intervals=[]
for _ in range(n):
    a,b =map(int,input().split())
    intervals.append([a,b])
if n<=0:
    print("Invalid input")
else:
    intervals.sort()
    res = []
    cur_start,cur_end = intervals[0]
    for i in range(1,n):
        s,e=intervals[i]
        if s<=cur_end:
            if e>cur_end:
                cur_end =e
        else:
            res.append([cur_start,cur_end])
            cur_start,cur_end = s,e
    res.append([cur_start,cur_end])
    for x in res:
        print(x[0],x[1])

5) # Digital archive timestamp organisation
def m(a,l,m1,r):
    b=a[l:m1+1]
    c=a[m1+1:r+1]

```

```

i=j=0
k=l
while i<len(b) and j<len(c):
    if b[i]<=c[j]:
        a[k]=b[i]
        i+=1
    else:
        a[k]=c[j]
        j+=1
    k+=1
while i<len(b):
    a[k] = b[i]
    i+=1
    k+=1
while j<len(c):
    a[k] = c[j]
    j+=1
    k+=1
def s(a,l,r):
    if l<r:
        m1= (l+r)//2
        s(a,l,m1)
        s(a,m1+1,r)
        m(a,l,m1,r)
n=int(input())
a=list(map(int,input().split()))
if len(set(a))!=n:
    print("Invalid Input")
else:
    s(a,0,n-1)
    print(*a)

```

PRACTICE:

1) # merge two arrays and sort

```
n=int(input())
arr1 = list(map(int,input().split()))
m= int(input())
arr2 = list(map(int,input().split()))
merged = arr1+arr2
merged.sort()
print(merged)
```

2) # Student score arrangement

```
n=int(input())
scores=list(map(int,input().split()))
for x in scores:
    if x<0:
        print("Invalid Input")
        exit()
for i in range(1,n):
    key=scores[i]
    j=i-1
    while j>=0 and scores[j]>key:
        scores[j+1]=scores[j]
        j=j-1
    scores[j+1]=key
print(*scores)
```

3) # Space mission fuel analysis

```
n=int(input())
arr = list(map(int,input().split()))
for x in arr:
```

```

if x>5000:
    print("Invalid Input")
    exit()
for i in range(n):
    min=i
    for j in range(i+1,n):
        if arr[j]<arr[min]:
            min=j
    arr[i],arr[min]=arr[min],arr[i]
print(*arr)

```

4) # Linear search-artifact identification

```

n=int(input())
arr=list(map(int,input().split()))
k=int(input())
if len(arr)!=len(set(arr)):
    print("Invalid Input")
else:
    found= False
    for i in range(n):
        if arr[i]==k:
            print(i+1)
            found = True
            break
    if not found:
        print(-1)

```

5) # Autonomous vehicle route priorities

```

def quick_sort(arr):
    if len(arr)<=1:
        return arr
    pivot= arr[0]

```

```

    left= [x for x in arr[1:]if x<pivot]
    right=[x for x in arr[1:]if x>=pivot]
    return quick_sort(left)+[pivot]+quick_sort(right)
n= int(input())
arr= list(map(int,input().split()))
for x in arr:
    if x<=0:
        print("Invalid Input")
        exit()
sorted_arr = quick_sort(arr)
print(*sorted_arr)

```

HASING AND MAPS

WORKOUT:

1) # Student data hasing

C = int(input())

Q = int(input())

table = [None] * C

for _ in range(Q):

parts = input().split()

if parts[0] == "insert":

if len(parts) != 3:

print("Invalid Operation")

continue

roll = int(parts[1])

marks = int(parts[2])

```
index = roll % C
```

```
if table[index] is None:
```

```
    table[index] = (roll, marks)
```

```
    print("Inserted")
```

```
else:
```

```
    if table[index][0] == roll:
```

```
        table[index] = (roll, marks)
```

```
        print("Inserted")
```

```
elif parts[0] == "search":
```

```
    if len(parts) != 2:
```

```
        print("Invalid Operation")
```

```
        continue
```

```
roll = int(parts[1])
```

```
index = roll % C
```

```
if table[index] is not None and table[index][0] == roll:
```

```
    print(f"Marks: {table[index][1]}")
```

```
else:
```

```
    print("Not Found")
```

```
elif parts[0] == "delete":
```

```
    if len(parts) != 2:
```

```
        print("Invalid Operation")
```

```
        continue
```

```
roll = int(parts[1])
```

```
index = roll % C
```

```
if table[index] is not None and table[index][0] == roll:
```

```
    table[index] = None
```

```
    print("Deleted")
```

```
else:
```

```
    print("Not Found")
```

```
elif parts[0] == "display":
```

```
    for i in range(C):
```

```
        if table[i] is None:
```

```
            print(f"{i}: NULL")
```

```
        else:
```

```
            print(f"{i}: {table[i][0]} -> {table[i][1]}")
```

```
else:
```

```
    print("Invalid Operation")
```

```
2) # Sum af all unique elements
```

```
n = int(input())
```

```
arr = list(map(int, input().split()))
```

```
freq = {}
```

```
for num in arr:
```

```
    freq[num] = freq.get(num, 0) + 1
```

```
total = 0
```

```
for num in arr:
```

```
    if freq[num] == 1:
```

```
        total += num
```



```
print(total)
```

3) # Double hasing technique

```
def h1(key, C):
```

```
    return key % C
```

```
def h2(key, C):
```

```
    return 1 + (key % (C - 1))
```

```
def insert(table, C, key, value):
```

```
    step = h2(key, C)
```

```
    for i in range(C):
```

```
        index = (h1(key, C) + i * step) % C
```

```
        if table[index] is None:
```

```
            table[index] = (key, value)
```

```
            print("Inserted")
```

```
            return
```

```
    print("Full")
```

```
def display(table):
```

```
    for i in range(len(table)):
```

```
        if table[i] is None:
```

```
            print(f"{i}: NULL")
```

```
        else:
```

```
            print(f"{i}: {table[i][0]} -> {table[i][1]}")
```

```
C = int(input())
```

```
N = int(input())
```

```
table = [None] * C
```

```
for _ in range(N):
```

```
    key, value = map(int, input().split())
```

```
    insert(table, C, key, value)
```

```
display(table)
```

```
4) # Employee ID Tracking
```

```
DELETED_MARKER = (-1, "DELETED")
```

```
def insert(table, capacity, key, value):
```

```
    index = key % capacity
```

```
    for i in range(capacity):
```

```
        new_index = (index + i * i) % capacity
```

```
        if table[new_index] is None or table[new_index] == DELETED_MARKER or  
table[new_index][0] == key:
```

```
            table[new_index] = (key, value)
```

```
            print("Inserted")
```

```
            return
```

```
print("Full")
```

```
def search(table, capacity, key):
```

```
    index = key % capacity
```

```
    for i in range(capacity):
```

```
        new_index = (index + i * i) % capacity
```

```
if table[new_index] is None:
```

```
    print("Not Found")
```

```
    return
```

```
if table[new_index] == DELETED_MARKER:
```

```
    continue
```

```
if table[new_index][0] == key:
```

```
    print(f"Dept: {table[new_index][1]}")
```

```
    return
```

```
print("Not Found")
```

```
def delete(table, capacity, key):
```

```
    index = key % capacity
```

```
    for i in range(capacity):
```

```
        new_index = (index + i * i) % capacity
```

```
    if table[new_index] is None:
```

```
        print("Not Found")
```

```
        return
```

```
    if table[new_index] == DELETED_MARKER:
```

```
        continue
```

```
    if table[new_index][0] == key:
```

```
        table[new_index] = DELETED_MARKER
```

```
        print("Deleted")
```

```
        return
```

```
print("Not Found")
```

```
def display(table):
```

```
    for i in range(len(table)):
```

```
        if table[i] is None or table[i] == DELETED_MARKER:
```

```
            print(f"{i}: NULL")
```

```
        else:
```

```
            print(f"{i}: {table[i][0]} -> {table[i][1]}")
```

```
C = int(input())
```

```
Q = int(input())
```

```
table = [None] * C
```

```
for _ in range(Q):
```

```
    parts = input().split()
```

```
    if parts[0] == "insert":
```

```
        if len(parts) == 3 and parts[1].isdigit():
```

```
            emp_id = int(parts[1])
```

```
            dept = parts[2]
```

```
            insert(table, C, emp_id, dept)
```

```
        else:
```

```
            print("Invalid Operation")
```

```
    elif parts[0] == "search":
```

```
        if len(parts) == 2 and parts[1].isdigit():
```

```
            emp_id = int(parts[1])
```

```
            search(table, C, emp_id)
```

```
        else:
```

```
            print("Invalid Operation")
```

```
    elif parts[0] == "delete":
```

```

    if len(parts) == 2 and parts[1].isdigit():
        emp_id = int(parts[1])
        delete(table, C, emp_id)
    else:
        print("Invalid Operation")
elif parts[0] == "print":
    if len(parts) == 1:
        display(table)
    else:
        print("Invalid Operation")
else:
    print("Invalid Operation")

5) # Online Exam seat allocation system
class ExamSystem:
    def __init__(self, slots):
        self.slots = slots
        self.data = [None] * slots
        self.removed = ("", "")

    def get_position(self, rid):
        return rid % self.slots

    def assign(self, rid, name):
        pos = self.get_position(rid)
        start = pos

        while self.data[pos] is not None and self.data[pos] != self.removed:
            if self.data[pos][0] == rid:
                self.data[pos] = (rid, name)
                print("Assigned")
            return

```

```

        pos = (pos + 1) % self.slots
        if pos == start:
            print("Slots Full")
            return

        self.data[pos] = (rid, name)
        print("Assigned")

def search(self, rid):
    pos = self.get_position(rid)
    start = pos

    while self.data[pos] is not None:
        if self.data[pos] != self.removed and self.data[pos][0] == rid:
            print(f"Name: {self.data[pos][1]}")
            return

        pos = (pos + 1) % self.slots
        if pos == start:
            break

    print("Not Found")

def remove(self, rid):
    pos = self.get_position(rid)
    start = pos

    while self.data[pos] is not None:
        if self.data[pos] != self.removed and self.data[pos][0] == rid:
            self.data[pos] = self.removed
            print("Removed")
            return

```

```
pos = (pos + 1) % self.slots
```

```
if pos == start:
```

```
    break
```

```
print("Not Found")
```

```
def print_table(self):
```

```
    for i in range(self.slots):
```

```
        if self.data[i] is None or self.data[i] == self.removed:
```

```
            print(f"{i}: NULL")
```

```
        else:
```

```
            print(f"{i}: {self.data[i][0]} -> {self.data[i][1]}")
```

```
M = int(input())
```

```
S = int(input())
```

```
es = ExamSystem(S)
```

```
for _ in range(M):
```

```
    cmd = input().split()
```

```
    if cmd[0] == "assign":
```

```
        es.assign(int(cmd[1]), cmd[2])
```

```
    elif cmd[0] == "search":
```

```
        es.search(int(cmd[1]))
```

```
    elif cmd[0] == "remove":
```

```
        es.remove(int(cmd[1]))
```

```
    elif cmd[0] == "print":
```

```
        es.print_table()
```

```
    else:
```

```
        print("Invalid Operation")
```

PRACTICE

1) # Product price lookup

```
size = int(input())
```

```
P = int(input())
```

```
table = [None] * size
```

```
for _ in range(P):
```

```
    product_id, price = map(int, input().split())
```

```
    index = product_id % size
```

```
    if table[index] is None:
```

```
        table[index] = (product_id, price)
```

```
for i in range(size):
```

```
    if table[i] is None:
```

```
        print(f"{i}: NULL")
```

```
    else:
```

```
        print(f"{i}: {table[i][0]} -> {table[i][1]}")
```

2) # Dominant value identifier

```
n = int(input())
```

```
arr = list(map(int, input().split()))
```

```
freq = {}
```

```
for num in arr:
```

```
    freq[num] = freq.get(num, 0) + 1
```



```
max_freq = 0
```

```
result = None
```

```
for num in arr:
```

```
    if freq[num] > max_freq:
```

```
        max_freq = freq[num]
```

```
        result = num
```

```
print(result)
```

```
3) # Counting elements greater than threshold
```

```
n = int(input())
```

```
arr = list(map(int, input().split()))
```

```
t = int(input())
```

```
freq = {}
```

```
for num in arr:
```

```
    freq[num] = freq.get(num, 0) + 1
```

```
count = 0
```

```
for num in freq:
```

```
    if num > t:
```

```
        count += freq[num]
```

```
print(count)
```

```
4) # Smart seat reservation system
```

```
n = int(input())
```

```
s = int(input())
```

```
table = [None] * s
```

```
for _ in range(n):
```

```
    cmd = input().split()
```

```
    if cmd[0] == "reserve":
```

```
        seat = int(cmd[1])
```

```
        status = cmd[2]
```

```
        idx = seat % s
```

```
        start = idx
```

```
        while table[idx] is not None and table[idx][0] != seat:
```

```
            idx = (idx + 1) % s
```

```
            if idx == start:
```

```
                break
```

```
        if table[idx] is None or table[idx][0] == seat:
```

```
            table[idx] = (seat, status)
```

```
            print("Reserved")
```

```
        else:
```

```
            print("Full")
```

```
    elif cmd[0] == "check":
```

```
        seat = int(cmd[1])
```

```
        idx = seat % s
```

```
        start = idx
```

```
        found = False
```

```
while table[idx] is not None:
```

```
    if table[idx][0] == seat:
```

```
        print("Status:", table[idx][1])
```

```
        found = True
```

```
        break
```

```
    idx = (idx + 1) % s
```

```
    if idx == start:
```

```
        break
```

```
if not found:
```

```
    print("Available")
```

```
elif cmd[0] == "cancel":
```

```
    seat = int(cmd[1])
```

```
    idx = seat % s
```

```
    start = idx
```

```
    found = False
```

```
while table[idx] is not None:
```

```
    if table[idx][0] == seat:
```

```
        table[idx] = None
```

```
        print("Cancelled")
```

```
        found = True
```

```
        break
```

```
    idx = (idx + 1) % s
```

```
    if idx == start:
```

```
        break
```

```
if not found:
```

```
print("Available")
```

```
elif cmd[0] == "display":
```

```
    for i in range(s):
```

```
        if table[i] is None:
```

```
            print(f"{i}: NULL")
```

```
        else:
```

```
            print(f"{i}: {table[i][0]} -> {table[i][1]}")
```

```
else:
```

```
    print("Invalid Operation")
```

5) # Online product review stroage system

```
t = int(input())
```

```
n = int(input())
```

```
table = [[] for _ in range(n)]
```

```
for _ in range(t):
```

```
    cmd = input().split()
```

```
    if cmd[0] == "add":
```

```
        key = int(cmd[1])
```

```
        review = " ".join(cmd[2:])
```

```
        idx = key % n
```

```
        found = False
```

```
        for i in range(len(table[idx])):
```

```
            if table[idx][i][0] == key:
```

```
                table[idx][i] = (key, review)
```

```
                found = True
```

```
        break

    if not found:

        table[idx].append((key, review))

    print("Added")


elif cmd[0] == "get":

    key = int(cmd[1])

    idx = key % n

    found = False

    for k, v in table[idx]:

        if k == key:

            print("Review:", v)

            found = True

            break

    if not found:

        print("Not Found")


elif cmd[0] == "remove":

    key = int(cmd[1])

    idx = key % n

    found = False

    for i in range(len(table[idx])):

        if table[idx][i][0] == key:

            table[idx].pop(i)

            print("Removed")

            found = True

            break

    if not found:

        print("Not Found")


elif cmd[0] == "display":
```

```
for i in range(n):
```

```
    if not table[i]:
```

```
        print(f"{i}: NULL")
```

```
    else:
```

```
        s = " ".join(f"{k} -> {v}" for k, v in table[i])
```

```
        print(f"{i}: {s}")
```

```
else:
```

```
    print("Invalid Operation")
```